UNIVERSITAT DE BARCELONA

Facultat de Matemàtiques
i Informàtica

GRAU DE MATEMÀTIQUES

Treball final de grau

Categorical Semantics and Autonomous Theorem Proving

Autor: Carlos Fernández Lorán

Director: Dr. Carles Casacuberta and Dr. Joost J. Joosten

Realitzat a: Departament of
Mathematics and Computer Science

Barcelona, June 7, 2026

Contents

Abstract

We develop a categorical framework for autonomous theorem proving by translating the problem of type inhabitation into a geometric graph exploration problem. Our main contributions are threefold.

First, we prove the initiality conjecture for Generalized Algebraic Theories, establishing that the term model $C(U)$ is the initial object of the category of models of a theory U. Second, we extend the Calculus of Constructions with a GAT signature, construct its term model, and prove the corresponding initiality result, building on Streicher's Doctrines of Constructions. Third, we design an Expansion Algorithm that exhaustively constructs the categorical search space of terms in both settings, and propose a GNN-based Conjecturer-Prover architecture to guide and prune this search, overcoming the inherent combinatorial explosion. We demonstrate the algorithm on simply typed combinatory logic using implica, a custom Rust-based graph engine.

2020 Mathematics Subject Classification. 03B38, 18C10, 68T30, 03F50

Introduction

Our ultimate goal in this thesis is to translate the syntactical problem of theorem proving into a geometrical graph-exploration problem.

The idea of generating mathematical reasoning computationally is one of the most profound challenges in computer science. At the core of this pursuit lies the Curry-Howard correspondence, an isomorphism between logic and computation [12]: mathematical propositions are interpreted as types, and the proofs of propositions are viewed as terms (or programs) inhabiting those types. Consequently, the problem of proving a theorem is reduced to the problem of finding a well-typed term in a formal type theory.

Modern proof assistants, such as Rocq (formerly Coq) [6] and Lean [8] leverage this correspondence using rich type theories like the Calculus of Inductive Constructions [7] [20]. While these environments enabled the formalization of highly complex mathematics, fully autonomous theorem proving with such rich frameworks remains an incredibly difficult challenge. The primary obstacle is undecidability of type inhabitation in dependent type theories [10], and even for inhabited types, attempting to exhaustively construct a term of a given type results in a combinatorial explosion of the search space.

Traditionally, term generation has been approached from a purely syntactic perspective, and more recently through Large Language Models. However, generating terms from syntax alone is difficult to guide heuristically, and the linear nature of language model generation forces early commitments that can prevent a proof from ever being found. Similar limitations have been observed by the Logical Intelligence team in [16].

This work grew out of an attempt to understand the framework proposed by Laurent Lafforgue, which connects Topos Theory [15] with artificial intelligence [13] [14]. While studying this connection, we found that Toposes, despite offering a strong theoretical foundation, were too abstract to serve directly as a practical representation for our purposes. This led us to categorical semantics as a more tractable alternative. Our central idea is to leverage the categorical semantics of type theories to reframe autonomous theorem proving as a structural graph ex-

ploration problem. By translating the syntax of Generalized Algebraic Theories (GATs) [4] [5] and the Calculus of Constructions (CoC) [6] into categorical structures —namely Contextual Categories and Doctrines of Constructions [21]— we obtain term models that carry a rich geometric graph structure. The soundness of this translation is guaranteed by the term model being the initial object of the category of models of the underlying type theory, a result known as the initiality conjecture, due to Voevodsky [24]. It was first proved by Streicher [21] for CoC, and later for full Martin-LÃ¶f type theory by Brunerie and Lumsdaine [3].

Within this categorical framework, the problem of finding a proof translates to finding a specific section —a specific morphism— withing a relativized contextual category. With these idea in mind of using categories and neural networks, we formalized an expansion algorithm —a deterministic process that exhaustively builds the finite subcategories representing the search space of terms— specifically designed to work together with Graph Neural Networks (GNNs), and try to overcome these intrinsic severe exponential growth inherent to any algorithm trying to exhaustivity solve the problem.

Graph Neural Networks are used to navigate and prune the categorical search space. Considering our finite subcategories that form the known search space as graphs, they are the perfect inputs for message-passing neural networks. By training a GNN to classify nodes in the context graph (deciding which paths to expand, keep or delete), we can effectively guide the expansion algorithm, bypassing the combinatorial explosion and paving the way for efficient, autonomous theorem proving.

Objectives

The main goals of this work are the following. In this work we first aim to provide a proof of the initiality conjecture for GATs, together with an extension of Streicher's proof for CoC together with a GAT signature. Then, once established the mathematical significance of objects and morphisms in the term model, we then present the expansion algorithm, an algorithm to construct any object or morphism in a Contextual Category or Contextual Doctrine, and that is designed to work together with a graph neural network to efficiently solve the problem in hand. In order to do this we will extend the concept of relativization from Context Categories to Doctrines of Constructions. Then we implement the expansion algorithm for simply typed combinatory logic, as a simple example to give intuition about how the expansion process works. Finally we give a proposal of a GNN-based model thought to optimize term generation.

Thesis Outline

This thesis is structured to guide the reader from the abstract categorical foundations to the practical implementation of AI-guided theorem proving:

- **Chapter 1: Syntax and Semantics of Generalized Algebraic Theories.** We introduce Cartmell's Generalized Algebraic Theories and define their syntax and semantics, as they were originally presented in [4] [5] and incorporating some notions presented by Streicher in [21]. Then we provide a concept of model of a GAT U together with a proof of the initiality of the term model $C(U)$. Finally, define the problem of type inhabitation in categorical terms, and present the foundational Expansion Algorithm.

- **Chapter 2: Calculus of Constructions.** We step up the complexity to the Calculus of Constructions (CoC), introducing product types and the universe of propositions. We present the definition of CoC together with the construction of Doctrines of Constructions as models for CoC, given by Streicher in [21]. We extend the definition of CoC to include a GAT signature, define the new term model and prove its initiality. Finally, we extend our categorical problem definition and Expansion Algorithm to handle this richer syntax.

- **Chapter 3: Implementation of the Expansion Algorithm.** We bring the theory into practice. We demonstrate the raw Expansion Algorithm generating terms of Simply Typed Combinatory Logic ($\mathbf{CL}_{\to}$) using IMPLICA, a custom Rust-based graph engine. Finally, we outline the architecture for integrating Graph Neural Networks using a novel unsupervised "Conjecturer-Prover" training paradigm to optimize the expansion process.

Chapter 1

Generalized Algebraic Theories

In this chapter, we introduce Generalized Algebraic Theories (GATs), a framework for working with dependent types. GATs allow us to construct the concept of *contextual category*, the foundational categorical structure to deal with dependent types. We use contextual categories as a baseline for interpreting the more complex Calculus of Constructions in Chapter 2. Finally, we formalize the term generation problem and introduce the expansion algorithm, our approach to term generation.

1.1 Syntax of GATs

Generalized Algebraic Theories, originally presented by J. Cartmell in his thesis [4] [5], is an abstraction of Martin-Löf type theory and is a generalization of the usual notion of a many-sorted algebraic theory in the following manner: while in Martin Löf's many-sorted algebraic theories sorts are constant types, interpreted as sets, in generalized algebraic theories sorts might be constant or variable, over a specified range of types. Variable types are interpreted as families of sets. We will start by giving some intuition, and we will later give the formal definition.

Informally, a generalized algebraic theory consists of:

1. a set of sorts, that can be constant or variable;
2. a set of operators, with the sorts of the arguments and value specified;
3. a set of axioms, where each axiom is an identity between similar expressions.

To build intuition, consider the theory of categories. The set of sorts contains two elements *Ob* and Hom. *Ob* is a constant type and Hom is a variable type; for any two terms x, y of type *Ob*, we have a type $\mathrm{Hom}(x, y)$. Here, we see a key feature of GATs: types vary over terms, not over types. The set of operators also

has two elements *id* and $\circ$. The *id* operator has one argument x of type *Ob*, and returns a value of type $\mathrm{Hom}(x,x)$. This shows how the type of value of an operator might vary depending on the arguments. The $\circ$ operator (or composition) expects 5 arguments, three terms x,y,z of type *Ob*, one term f of type $\mathrm{Hom}(x,y)$, and one term g of type $\mathrm{Hom}(y,z)$. It returns a value of type $\mathrm{Hom}(x,z)$. Here we see that the type of an argument might also vary depending on a previous argument. Informally, instead of writing $\circ(x,y,z,f,g)$ we can write $\circ(f,g)$, as the first three arguments might be inferred from the last two. This notation is discussed in the original paper [5].

Now we introduce the notation we will be using when talking about terms and types. This notation will be formally defined in a later section. Given a set of variables V, we associate types to variables as required. When asserting that a term t is of type A, we will write $t\colon A$. But if t has variables $x_1,\ldots,x_n$ occurring in it, this assertion does not make sense unless we assign some types to $x_1,\ldots,x_m$. To do this we write

$$x_1\colon A_1,\ldots,x_n\colon A_n \vdash t\colon A$$

that reads as follows: if x_1 is a variable of type $A_1,\ldots$ and if x_n is a variable of type A_n, then t is a term of type A. Similarly, if we want to assert that if x_1 is a variable of type $A_1,\ldots$ and if x_n is a variable of type A_n, then A is a type, we can write:

$$x_1\colon A_1,\ldots,x_n\colon A_n \vdash A \text{ type}$$

Such assertions are called rules or sequents. Instead of considering expressions or judgments as basic units, we will use rules, as both expressions and judgments make sense only within a context.

Axioms are also written as rules. The conclusion of those rules are equalities between types or between terms of the same type. We write those rules as

$$x_1\colon A_1,\ldots,x_n\colon A_n \vdash A = A' \quad \text{and} \quad x_1\colon A_1,\ldots,x_n\colon A_n \vdash t = t'\colon A$$

that read as: if x_1 is a variable of type $A_1,\ldots$ and x_n is a variable of type A_n, then $A = A'$, or $t = t'$ as terms of type A, respectively. We will be using both shorthand and fraction notation as needed.

To continue with the previous example, we can write the axioms of category theory as follows:

$$\begin{aligned}
&x,y\colon Ob, f\colon Hom(x,y) \vdash \circ(id(x),f) = f,\\
&x,y\colon Ob, f\colon Hom(x,y) \vdash \circ(f,id(y)) = f,\\
&w,x,y,z\colon Ob, f\colon Hom(w,x), g\colon Hom(x,y), h\colon Hom(y,z)\\
&\qquad \vdash \circ(\circ(f,g),h) = \circ(f,\circ(g,h)).
\end{aligned}$$

When presenting a theory's language i.e., the sets of sorts and operators, we

must present each symbol with a rule specifying the role of this symbol, either as a sort symbol or as specifically typed operator symbol. These rules are called *introductory rules*. In the case of a sort symbol A, the rule should be like

$$x_1 \colon A_1, \ldots, x_n \colon A_n \vdash A(x_1, \ldots, x_n) \text{ type}$$

clearly specifying the types A is dependent on. In the case of an operator symbol, a rule of the form

$$x_1 \colon A_1, \ldots, x_n \colon A_n \vdash F(x_1, \ldots, x_n) \colon A$$

suffices to specify the types of the arguments and the type of the value of F.

Finally, to present a theory we will give a set of symbols, each associated to its introductory rule, and a set of axioms. And, of course, everything must be well-formed, something we will address shortly. Now we give a formal presentation of category theory using GAT notation: in Table 1.1 we state the symbols of the theory together with their introductory rules and in Table 1.2 we state the axioms of the theory as rules.

Symbol	*Introductory Rule*
Ob	$\vdash Ob$ type
Hom	$x, y \colon \mathrm{Ob} \vdash \mathrm{Hom}(x, y)$ type
$\circ$	$x, y, z \colon Ob, f \colon \mathrm{Hom}(x, y), g \colon \mathrm{Hom}(y, z)$ $\vdash \circ(f, g) \colon \mathrm{Hom}(x, z)$
id	$x \in Ob \vdash id(x) \colon \mathrm{Hom}(x, x)$

Table 1.1: Symbols and their introductory rules.

Axioms
$x, y \colon Ob, f \colon \mathrm{Hom}(x, y) \vdash \circ(id(x), f) = f$
$x, y \colon Ob, f \colon \mathrm{Hom}(x, y) \vdash \circ(f, id(y)) = f$
$w, x, y, z \colon Ob, f \colon \mathrm{Hom}(w, x), g \colon \mathrm{Hom}(x, y), h \colon \mathrm{Hom}(y, z)$ $\vdash \circ(\circ(f, g), h) = \circ(f, \circ(g, h))$

Table 1.2: Axioms (left unit, right unit, associativity).

Now we formally define all of the concepts we just introduced. To give a notion of well-formed introductory rule, we need the concept of derivability, which also depends on the introductory rules. To tackle this problem, we leave the question of well-formedness aside until we have the complete set of derived rules of a theory. We first define a pre-theory and then define a theory as a well-formed pre-theory.

We assume given a countable set of variables V. Let us define the notion of rule on an alphabet W, whose elements are called symbols.

Definition 1.1. The set of *expressions* $\mathcal{E}$, is defined inductively so that expressions are finite sequences of elements of $W \cup V \cup \{(\} \cup \{)\} \cup \{,\}$ defined by the following clauses:

1. if $x \in V$, then x is an expression;
2. if $L \in W$, then L is an expression;
3. if $L \in W$ and $e_1, \ldots, e_n$ are expressions, then $L(e_1, \ldots, e_n)$ is an expression.

Now we can define the building blocks of a rule:

Definition 1.2. A *premise* is defined to be a sequence of elements of $V \times \mathcal{E}$. The empty sequence is allowed and we call it an empty premise. We denote the premise $((x_1, A_1), \ldots, (x_n, A_n))$ as $x_1 \colon A_1, \ldots, x_n \colon A_n$.

Definition 1.3. We call a *conclusion* to one of the following:

1. T-conclusion: defined by a single expression A and written as 'A type'.
2. $\in$-conclusion: defined by a pair of expressions (t, A) and written as '$t \colon A$'.
3. T $=$conclusion: defined by a pair of expressions (A_1, A_2) and written as '$A_1 = A_2$'.
4. $\in=$conclusion: defined by a triple of expressions (t_1, t_2, A) and written as '$t_1 = t_2 \colon A$'

Finally a *rule* is determined by a premise P and a conclusion C, and is written as $P \vdash C$. We name the rule as T-rule, $\in$-rule, T $=$rule or $\in=$rule according to the form of the conclusion.

Now we can define a *pre-theory* and later a *theory*.

Definition 1.4. A *pre-theory* consists of

1. a set of sort symbols $\mathcal{S}$;
2. a set of operator symbols $\mathcal{Z}$;

3. for each sort symbol $A \in \mathcal{S}$, an associated rule of the alphabet $\mathcal{S} \cup \mathcal{Z}$:

$$x_1 \colon A_1, \ldots, x_n \colon A_n \vdash A(x_1, \ldots, x_n) \text{ type}$$

for some $n \geq 0$. It is called the *introductory rule* of A;

4. for each operator symbol $F \in \mathcal{Z}$, an associated rule of the alphabet $\mathcal{S} \cup \mathcal{Z}$:

$$x_1 \colon A_1, \ldots, x_n \colon A_n \vdash F(x_1, \ldots, x_n) \colon A$$

for some $n \geq 0$. It is called the *introductory rule* of F;

5. a set of axioms. Where each axiom is either a T =rule or a $\in$=rule of the alphabet $S \cup Z$.

Definition 1.5. A pre-theory is *well-formed* if all of its introductory rules are well-formed. A *theory* is a well-formed pre-theory.

Now we tackle the problem of well-formedness of rules.

Definition 1.6. If U is a pre-theory, then

1. a *context* is a premise $x_1 \colon A_1, \ldots, x_n \colon A_n$ such that the rule

$$x_1 \colon A_1, \ldots, x_{n-1} \colon A_{n-1} \vdash A_n \text{ type}$$

is a derived rule such that x_n is distinct from all of $x_1, \ldots, x_n$.

2. (a) the rule $\Gamma \vdash A$ type is *well-formed* if the premise Γ is a context.
   (b) the rule $\Gamma \vdash t \colon A$ is *well-formed* if $\Gamma \vdash A$ type is a derived rule of U.
   (c) the rule $\Gamma \vdash A = A'$ is *well-formed* if $\Gamma \vdash A$ type and $\Gamma \vdash A'$ type are derived rules of U.
   (d) the rule $\Gamma \vdash t = t' \colon A$ is *well-formed* if $\Gamma \vdash t \colon A$ and $\Gamma \vdash t' \colon A$ are derived rules of U.

Definition 1.7. The set of *derived rules* of a pre-theory U is the set of rules derivable by the derivation rules presented in Appendix A.1.

Lastly, we introduce *substitution*. Given expressions $\Delta, t_1, \ldots, t_n$ and variables $x_1, \ldots, x_n$, then we define the expression $\Delta[x_1 \leftarrow t_1, \ldots, x_n \leftarrow t_n]$ as the result of simultaneous replacement of $x_1, \ldots, x_n$ in Δ by $t_1, \ldots, t_n$. We call this the substitution operation. Then, as proven in the original paper [5], the set of derived rules of a theory is closed under the operation of substitution of correctly typed terms for variables, i.e., that we can substitute a variable x of type A in an expression only by terms t such that we can derive that $t \colon A$. We could add substitution as one of the derivation rules, but that would make applying induction very messy.

To transition from syntax to geometry (which our Graph Neural Network will eventually navigate), we construct a category [17] out of the theory syntax.

Definition 1.8. For a theory U, we define the *category of contexts and realizations*, denoted $R(U)$. This category has as objects the contexts of U. Given two contexts $\Gamma = x_1 \colon A_1, \ldots, x_n \colon A_n$ and $\Delta = y_1 \colon B_1, \ldots, y_m \colon B_m$ of U, a morphism from Γ to Δ, or a *realization* of Δ from Γ, is an m-tuple $\langle t_1, \ldots, t_m \rangle$ such that for each j, $1 \leq j \leq m$, the rule

$$\Gamma \vdash t_j \colon B_j[y_1 \leftarrow t_1, \ldots, y_{j-1} \leftarrow t_{j-1}]$$

is derivable. We define the identity of an object $x_1 \colon \Delta_1, \ldots, x_n \colon \Delta_n$ is the realization $\langle x_1, \ldots, x_n \rangle$, and composition is defined as

$$\langle t_1, \ldots, t_m \rangle \circ \langle s_1, \ldots, s_w \rangle = \\ \langle s_1[y_1 \leftarrow t_1, \ldots, y_m \leftarrow t_m], \ldots, s_w[y_1 \leftarrow t_1, \ldots, y_m \leftarrow t_m] \rangle$$

for $\Gamma = x_1 \colon A_1, \ldots, x_n \colon A_n$, $\Delta = y_1 \colon B_1, \ldots, y_m \colon B_m$ and $\Omega = z_1 \colon C_1, \ldots, z_w \colon C_w$ contexts, and $\langle t_1, \ldots, t_m \rangle \colon \Gamma \to \Delta$ and $\langle s_1, \ldots, s_w \rangle \colon \Delta \to \Omega$ realizations.

Now we study some properties of this category. First, we see that it has a tree structure.

Definition 1.9. A tree is a pair (N, parent) where N is a nonempty set of nodes and $\text{parent} \colon N \to N$ is a function such that:

1. for all $A \in N$, there exists $n \in \omega$ such that $\text{parent}^n(A) = \text{parent}^{n+1}(A)$;
2. for all $A, B \in N$, if $A = \text{parent}(A)$ and $B = \text{parent}(B)$, then $A = B$.

We write $A \trianglelefteq B$ if $A = \text{parent}(B)$ and $A \triangleleft B$ if $A \trianglelefteq B$ and $A \neq B$. By Condition 2, there is a unique least element of N, which we call 1. Finally, we will define, for any $A \in N$, $\text{level}(A)$ to be the least $n \in \omega$ with $\text{parent}^n(A) = \text{parent}^{n+1}(A)$. We will consider $\leq$ to be the transitive completion of the relation $\trianglelefteq$. We can see that the set of objects of $R(U)$, i.e., the set of contexts of U, is structured as a tree with the empty context as its least element and $\text{parent}(\langle x_1 \colon A_1, \ldots, x_n \colon A_n \rangle) = \langle x_1 \colon A_1, \ldots, x_{n-1} \colon A_{n-1} \rangle$. Moreover, given a context $x_1 \colon A_1, \ldots, x_n \colon A_n, x_{n+1} \colon A_{n+1}, \ldots x_{n+m}$ of U, $\langle x_1, \ldots, x_n \rangle$ is a realization of $x_1 \colon A_1, \ldots, x_n \colon A_n$ with regard to $x_1 \colon A_1, \ldots, x_n \colon A_n, x_{n+1} \colon A_{n+1}, \ldots, x_{n+m} \colon A_{n+m}$, and is noted as $p(\langle x_1 \colon A_1, \ldots, x_{n+m} \colon A_{n+m} \rangle, \langle x_1 \colon A_1, \ldots, x_n \colon A_n \rangle)$. Hence, if we consider $\Gamma, \Delta \in ObR(U)$ such that $\Gamma \leq \Delta$, then $p(\Delta, \Gamma)$ is defined and $p(\Delta, \Gamma) \colon \Delta \to \Gamma$ in $R(U)$.

Additionally, for contexts $\Gamma = x_1 \colon A_1, \ldots, x_n \colon A_n$, $\Gamma' = y_1 \colon B_1, \ldots, y_m \colon B_m$ and $\Delta = y_1 \colon B_1, \ldots, y_{m+w} \colon B_{m+w}$, and morphism $f = \langle t_1, \ldots, t_m \rangle \colon \Gamma \to \Gamma'$ in $R(U)$,

$$\begin{array}{ccc} f^*\Delta & \xrightarrow{q(f,\Delta)} & \Delta \\ {\scriptstyle p(f^*\Delta,\Gamma)}\downarrow & & \downarrow{\scriptstyle p(\Delta,\Gamma')} \\ \Gamma & \xrightarrow{f} & \Gamma' \end{array}$$

is a pullback diagram [17] in $R(U)$, where

$$f^*\Delta = x_1 : A_1, \ldots, x_n : A_n,$$
$$y_{m+1} : B[y_1 \leftarrow t_1, \ldots, y_m \leftarrow t_m], \ldots, y_{m+w} : B[y_1 \leftarrow t_1, \ldots, y_m \leftarrow t_m]$$
$$q(f, \Delta) = \langle t_1, \ldots, t_m, y_{m+1}, \ldots, y_{m+w} \rangle.$$

This motivates the following definition:

1.2 Contextual Categories

Definition 1.10. A *contextual category* consists of

1. a category $\mathcal{C}$ with a terminal object 1;
2. a function parent: $Ob\mathcal{C} \to Ob\mathcal{C}$ such that $(Ob\mathcal{C}, \text{parent})$ is a tree, and 1 is the root of the tree, i.e., parent$1 = 1$;
3. for each object A of $\mathcal{C}$ different from 1, a morphism $p(A) : A \to \text{parent}A$, called *canonical projection of* A, which we write as $A \twoheadrightarrow \text{parent}(A)$;
4. for all objects A, B in $\mathcal{C}$ with $A \lhd B$ and all morphisms $f : C \to A$, an object f^*B of $\mathcal{C}$ such that $C \lhd f^*B$, and a morphism $q(f, B) : f^*B \to B$ in $\mathcal{C}$ such that

$$\begin{array}{ccc} f^*B & \xrightarrow{q(f,B)} & B \\ \downarrow & & \downarrow \\ C & \xrightarrow{f} & A \end{array}$$

is a pullback in $\mathcal{C}$ that satisfies the following *functoriality conditions*:

(a) if A and B are objects in $\mathcal{C}$ such that $A \lhd B$, then

$$id_A^*B = B \quad \text{and} \quad q(id_A, B) = id_B;$$

(b) whenever

$$\begin{array}{ccccc} A & \xrightarrow{f} & A' & \xrightarrow{f'} & A'' \\ & & & & \downarrow \\ & & & & B \end{array}$$

in $\mathcal{C}$, then

$$(f' \circ f)^*B = f^*(f'^*B) \quad \text{and} \quad q(f' \circ f, B) = q(f', B) \circ q(f, f'^*B).$$

We have that $R(U)$ is structured as a contextual category. The problem with $R(U)$ is that semantically equal but syntactically different rules are interpreted as

different elements. Hence we define an equivalence relation $\equiv$ between derived T- and $\in$- rules as follows:

$$x_1 : A_1, \ldots, x_n : A_n \vdash A_{n+1} \text{ type} \equiv y_1 : B_1, \ldots, y_m : B_m \vdash B_{m+1} \text{ type}$$

if and only if $m = n$ and, for each $1 \leq i \leq n+1$,

$$x_1 : A_1, \ldots, x_{i-1} : A_{i-1} \vdash A_i = B_i[y_1 \leftarrow x_1, \ldots, y_{i-1} \leftarrow x_{i-1}]$$

is derivable, and

$$x_1 : A_1, \ldots, x_n : A_n \vdash t : A \equiv y_1 : B_1, \ldots, y_m : B_m \vdash s : B$$

if and only if

$$x_1 : A_1, \ldots, x_n : A_n \vdash A \text{ type} \equiv y_1 : B_1, \ldots, y_m : B_m \vdash B \text{ type}$$

and

$$x_1 : A_1, \ldots, x_n : A_n \vdash t = s[y_1 \leftarrow x_1, \ldots, y_m \leftarrow x_m] : A$$

is derivable.

We define the *context category* of U, denoted by $C(U)$, as the category of equivalence classes of contexts and of equivalence classes of realizations. Two contexts $x_1 : A_1, \ldots, x_n : A_n$ and $y_1 : B_1, \ldots, y_m : B_m$ are equivalent if and only if

$$x_1 : A_1, \ldots, x_{n-1} : A_{n-1} \vdash A_n \text{ type} \equiv y_1 : B_1, \ldots, y_{m-1} : B_{m-1} \vdash B_m \text{ type}$$

and two realizations

$$\langle t_1, \ldots, t_m \rangle : \langle x_1 : A_1, \ldots, x_n : A_n \rangle \to \langle y_1 : B_1, \ldots, y_m : B_m \rangle$$
$$\langle s_1, \ldots, s_m \rangle : \langle x'_1 : A'_1, \ldots, x'_n : A'_n \rangle \to \langle y'_1 : B'_1, \ldots, y'_m : B'_m \rangle$$

are equivalent if and only if, for each j, $1 \leq j \leq n$,

$$x_1 : A_1, \ldots, x_n : A_n \vdash t_j : B_j[y_1 \leftarrow t_1, \ldots, y_{j-1} \leftarrow t_{j-1}] \equiv$$
$$x'_1 : A'_1, \ldots, x'_n : A'_n \vdash s_j : B'_j[y'_1 \leftarrow s_1, \ldots, y_{j-1} \leftarrow s_{j-1}].$$

It follows that $C(U)$ is itself a contextual category.

Definition 1.11. If $\mathcal{C}$ and $\mathcal{C}'$ are contextual categories, then a **contextual functor** is a functor [17] $F : \mathcal{C} \to \mathcal{C}'$ such that:

1. $F(1) = 1$ and if $A \lhd B$ in $\mathcal{C}$, then $F(A) \lhd F(B)$ in $\mathcal{C}'$;
2. for every object A of $\mathcal{C}$, $F(p(A)) = p(F(A))$;
3. for all f and B such that f^*B is defined in $\mathcal{C}$,

$$F(f^*B) = F(f)^*F(B) \quad \text{and} \quad F(q(f,B)) = q(F(f), F(B)).$$

Con denotes the category of contextual categories and contextual functors.

Now, we will address some important properties of contextual categories we will be using later.

The first useful result is about projections; Although we defined projections on pairs $A \lhd B$, we might extend the notion to pairs $A \leq B$ defined as

$$p(B, A) = p(X_1) \circ \cdots \circ p(X_n) \circ p(B),$$

where $X_1, \ldots, X_n$ is the unique sequence of objects in $\mathcal{C}$ such that $A \lhd X_1 \lhd \cdots \lhd X_n \lhd B$ in $\mathcal{C}$. In the case where $A = B$, we take $p(B, A) = id_A$. Notation: $B \longrightarrow\!\!\!\!\!\triangleright\ A$.

In type theory, proving a theorem often requires working under a set of assumptions (a context). In categorical semantics, "assuming a context A" corresponds to taking the relativization of $\mathcal{C}$ to A, noted as $\mathcal{C}_A$. More formally, let $\mathcal{C}$ be a contextual category and A an arbitrary object of $\mathcal{C}$. Then we define the *relativization* of $\mathcal{C}$ to A as the contextual category where:

1. The objects of $\mathcal{C}_A$ are the objects B of $\mathcal{C}$ such that $A \leq B$.
2. For objects B, C in $\mathcal{C}_A$, a morphism from B to C in $\mathcal{C}_A$, also referenced as a morphism from B to C over A, is a morphism $f : B \to C$ in $\mathcal{C}$ such that $p(C, A) \circ f = p(B, A)$. Composition of morphisms is inherited from $\mathcal{C}$.
3. $\text{parent}_A(B) = \text{parent}(B)$ if $A < B$, and $\text{parent}_A(A) = A$.
4. For objects $B > A$, $p_A(B) = p(B) : B \to \text{parent}(B)$.
5. For B, C, D objects of $\mathcal{C}_A$ with $\text{parent}_A(D) = C$ and morphisms $f : B \to C$, we write
$$f^{*_A} D = f^* D \quad \text{and} \quad q_A(f, D) = q(f, D).$$

Now we see that any morphism $f : B \to A$ in $\mathcal{C}$ induces a pullback functor $f^* : \mathcal{C}_A \to \mathcal{C}_B$ which is a contextual functor. The proof of the following theorems is included in [21]. This functor can be thought as a way of embedding the knowledge over the context A onto the context B.

Lemma 1.12. *Let $\mathcal{C}$ be a contextual category, and $f : B \to A$ a morphism in $\mathcal{C}$. Then, for any object D with $D \geq A$, we define an object $f^*D \geq B$ and a morphism $q(f, D) : f^*D \to D$ by induction over $level D - level A$:*

1. *if $D = A$,*
$$f^* D = B \quad \textit{and} \quad q(f, D) = f;$$
2. *if $D > A$,*
$$f^* D = q(f, parent(D))^* D \quad \textit{and} \quad q(f, D) = q(q(f, parent(D)), D).$$

Then, if $D \geq A$, the pair $(p(f^*D, B), q(f, D))$ is a pullback cone over $(f, p(D, A))$.

Theorem 1.13. *Let $\mathcal{C}$ be a contextual category and $f\colon B \to A$ a morphism in $\mathcal{C}$. By mapping an object C of $\mathcal{C}_A$ to f^*C in $\mathcal{C}_B$, and mapping $g\colon C \to D$ over A to $f^*g\colon f^*C \to f^*D$ over B determined by*

$$p(f^*D, B) \circ f^*g = p(f^*C, B)$$
$$q(f, D) \circ f^*g = q(f, C)$$

we obtain a contextual functor $f^*\colon \mathcal{C}_A \to \mathcal{C}_B$ called reindexing along f.

Theorem 1.14. *Let $\mathcal{C}$ be a contextual category and $f\colon B \to A$, $g\colon C \to B$ be morphisms in $\mathcal{C}$. Then,*

$$g^* \circ f^* = (f \circ g)^* \quad \text{and} \quad q(f \circ g, D) = q(f, D) \circ q(g, f^*D)$$

for objects $D \geq A$.

Definition 1.15. [21] Let $\mathcal{C}$ be a contextual category, A and B be objects of $\mathcal{C}$ such that $A \lhd B$. The set of **sections** of B is defined as

$$\text{Sections}(B) \coloneqq \{s\colon A \to B \leftarrow p(B) \circ s = id_A\}.$$

If $\text{level}(B) = 1$, then

$$\text{Sections}(B) = \{s \leftarrow s\colon 1 \to B\},$$

as any morphism $s\colon 1 \to B$ satisfies $p(B) \circ s = id_1$ since 1 is a terminal object. In this case, the notion of section matches the notion of *global element* of B. The concept of section of a context $\langle x_1\colon A_1, \ldots, x_n\colon A_n, y\colon B\rangle$ is the categorical equivalent of a term of type B under context $\langle x_1\colon A_1, \ldots, x_n\colon A_n\rangle$.

1.3 Relevance of the Context Category

The context category is the first example of what it is called a **term model**. This means that it is a category built directly from the syntax of a theory without picking any particular mathematical interpretation. This gives us the *most generic* model of that theory. The three main properties of this models are the following:

1. **Soundness**: Any model of such theory corresponds to a structure preserving functor out of the term model.

2. **Completeness**: A judgement is provable/derivable in the theory if it holds in all models, or equivalently if it holds in the term model.

3. **Initiality**: The term model is often the initial object of the category of all models.

Because of completeness, navigating $C(U)$ is equivalent to manipulating the syntax of U.

We start by proving the completeness of $C(U)$. The following completeness theorem is direct from the definition of $C(U)$:

Theorem 1.16 (Completeness Theorem)**.** *Let $\mathcal{M}$ be the canonical interpretation of a theory U in $C(U)$.*

1. *The T-rule $\Gamma \vdash A$ type is derivable if and only if $\Gamma, x\colon A$ is a context, i.e., if $[\Gamma, x\colon A]$ is an object of $C(U)$.*

2. *The $\in$-rule $\Gamma \vdash t\colon A$, for $\Gamma = x_1\colon A_1, \ldots, x_n\colon A_n$, is derivable if and only if $[\langle x_1, \ldots, x_n, t\rangle]$ is a section of $[\Gamma, x\colon A]$ in $C(U)$.*

3. *If the T-rules $\Gamma \vdash A_1$ type and $\Gamma \vdash A_2$ type are derivable, then the rule $\Gamma \vdash A_1 = A_2$ is derivable if and only if $[\Gamma, x\colon A_1] = [\Gamma, x\colon A_2]$, where x is a variable that does not appear in Γ.*

4. *If the $\in$-rules $\Gamma \vdash t\colon A$ and $\Gamma \vdash s\colon A$, for $\Gamma = x_1\colon A_1, \ldots, x_n\colon A_n$, are derivable, then the rule $\Gamma \vdash t = s\colon A$ is derivable if, and only if, $\langle x_1, \ldots, x_n, t\rangle$ and $\langle x_1, \ldots, x_n, s\rangle$ are equal sections of $[\Gamma, x\colon A]$ in $C(U)$, where x is a variable not appearing in Γ.*

Now we prove the initiality of $C(U)$, a special case of Voevodsky's initiality conjecture [24]. Hence, we need to define interpretation of a theory U in a contextual category, and see that the interpretation into $C(U)$ is the initial object of the category of interpretations of U.

Given a contextual category $\mathcal{C}$ and a theory U, an *interpretation* of U in $\mathcal{C}$ is a mapping $\mathcal{M}$ from the T- and $\in$-rules of U to the $\mathcal{C}$ such that

1. if R is a derived T-rule $x_1\colon A_1, \ldots, x_n\colon A_n \vdash A$ type and we name the rules $x_1\colon A_1, \ldots, x_{i-1}\colon A_{i-1} \vdash A_i$ type as R_i for $1 \leq i \leq n$, then $\mathcal{M}[R_1], \ldots, \mathcal{M}[R_n]$, $\mathcal{M}[R]$ are objects of $\mathcal{C}$ such that $1 \lhd \mathcal{M}[R_1] \lhd \cdots \lhd \mathcal{M}[R_n] \lhd \mathcal{M}[R]$ in $\mathcal{C}$.

2. if R_t is a derived $\in$-rule $x_1\colon A_1, \ldots, x_n\colon A_n \vdash t\colon A$ then $\mathcal{M}[R_t]$ is section of $\mathcal{M}[R]$.

3. if $\Gamma \vdash A = A'$ is a derived T =rule, then if we name $\Gamma \vdash A$ type as R and $\Gamma \vdash A'$ type as R', then $\mathcal{M}[R] = \mathcal{M}[R']$.

4. if $\Gamma \vdash t = t' : A$ is a derived $\in$=rule, then if we name $\Gamma \vdash t : A$ as R_t, $\Gamma \vdash t' : A$ as $R_{t'}$ and $\Gamma \vdash A$ type as R, then both $\mathcal{M}[R_t]$ and $\mathcal{M}[R_{t'}]$ are sections of $\mathcal{R}$. and $\mathcal{M}[R_t] = \mathcal{M}[R_{t'}]$

There is a canonical interpretation $\mathcal{M}$ of U in $\mathcal{C}(U)$, defined as follows:

1. if $R \equiv x_1 : A_1, \ldots, x_n : A_n \vdash A$ type is a derived T-judgement, then we define
$$\mathcal{M}[R] = [x_1 : A_1, \ldots, x_n : A_n, x_{n+1} : A];$$
2. if $R_t \equiv x_1 : A_1, \ldots, x_n : A_n \vdash t : A$ is a derived $\in$-judgement, then we define
$$\mathcal{M}[R_t] = [\langle x_1, \ldots, x_n, t \rangle]$$
a section of $\mathcal{M}[R]$.

The Completeness Theorem 1.16 guarantees that $\mathcal{M}$ is well defined.

Now we shall define the concept of homomorphism between interpretations of a theory U. Let $\mathcal{C}, \mathcal{C}'$ be two contextual categories, and let $\mathcal{M}_\mathcal{C}$ and $\mathcal{M}_{\mathcal{C}'}$ be two U-interpretations in $\mathcal{C}$ and $\mathcal{C}'$ respectively. Then, an interpretation homomorphism is a contextual functor $F : \mathcal{C} \to \mathcal{C}'$ such that:

1. if R is a T-rule, then $F\mathcal{M}_\mathcal{C}[R] = \mathcal{M}_{\mathcal{C}'}[R]$;
2. if R_t is a $\in$-rule, then $F\mathcal{M}_\mathcal{C}[R_t] = \mathcal{M}_{\mathcal{C}'}[R_t]$.

So we can define the category $\mathbf{Con}_U$ as the category with pairs of contextual categories and U-interpretations and their homomorphisms. We can easily see that $(C(U), \mathcal{M})$ is an initial object of $\mathbf{Con}_U$. Consider an object $(\mathcal{C}, \mathcal{N})$ of $\mathbf{Con}_U$. Let construct a morphism $F : ((C(U), \mathcal{M})) \to (\mathcal{C}, \mathcal{N})$, i.e., we have to construct a contextual functor $F : C(U) \to \mathcal{C}$ that *commutes* with the interpretation functions. If $[\Gamma]$ an object of $C(U)$, then there is a T-rule R^Γ such that $\mathcal{M}[R^\Gamma] = \Gamma$. We define $F\Gamma = \mathcal{N}[R^\Gamma]$. In order to define F on the morphisms of $C(U)$, it suffices to define it for the sections of $C(U)$, as any morphism
$$\langle t_1, \ldots, t_m \rangle : \langle x_1 : A_1, \ldots, x_n : A_n \rangle \to \langle y_1 : B_1, \ldots, y_m : B_m \rangle$$
can be built by composition and pullback functors of sections and projections, and the latter is uniquely determined by the condition that F is a contextual functor. Hence, given a section $s = \langle x_1, \ldots, x_n, t \rangle$ of $R \equiv x_1 : A_1, \ldots, x_n : A_n, x : A$, by definition there exists a derived $\in$-rule R_t such that $\mathcal{M}[R_t] = s$, so we can define $Fs = \mathcal{N}[R_t]$. The uniqueness of F is rather simple to see, as we cannot assign any other values to any object or morphism of $C(P)$ without conflicting with the structural conditions of the functor. Hence, $(C(U), \mathcal{M})$ is an initial object of $\mathbf{Con}_U$.

Soundness comes from the initiality of $C(U)$, as any model of U, i.e., any element of $\mathbf{Con}_U$, is equivalent to the initial morphism F we just defined.

1.4 Expansion Algorithm for GATs

In this section, we give formal definition of our problem, proposed at the beginning of the thesis, and reformulate it to an equivalent problem easier to solve. Then we propose an algorithm to solve such problem, that is designed to work together with artificial intelligence. Such optimization is discussed in Chapter 3.

Our problem might be formalized in the following manner: Given a theory U and a derived rule of U of the form

$$\frac{x_1 \colon A_1, \ldots, x_n \colon A_n}{A \text{ type}}$$

we want to find an expression t of U such that the rule

$$\frac{x_1 \colon A_1, \ldots, x_n \colon A_n}{t \colon A}$$

is a derived rule of U.

This problem is undecidable, in general, as it follows from the undecidability of the inhabitation in a GAT. This is why we will not try to find an algorithm that either finds a term or says that the type is empty. We aim to find an algorithm that, in case that the type is inhabited, it finds a term in a finite number of steps. In case that the type is empty, it does not have to give an output.

We can consider this problem from the point of view of the context category $C(U)$, where by the Completeness Theorem 1.16 it is rephrased as follows: Given a context $\Gamma = x_1 \colon A_1, \ldots, x_n \colon A_n$ in $C(U)$, we want to find a section of $\Gamma, x \colon A$ in $C(U)$.

We can restrict our search to the *relativization* of $C(U)$ to $[\Gamma]$, that we denote as $\mathcal{C}_\Gamma$. We denote a morphism $\langle x_1, \ldots, x_n, t_{n+1}, \ldots, t_{n+m} \rangle$ in $\mathcal{C}_\Gamma$, as $\langle t_{n+1}, \ldots, t_{n+m} \rangle_\Gamma$.

Now we define the *expansion algorithm* that will be responsible of finding $\langle t \rangle_\Gamma$ in $\mathcal{C}_\Gamma$. The complexity of this algorithm is really high, and it will serve as a baseline to be optimized by a graph neural network. We will explain this in the last chapter.

First, consider a finite subcategory G of $\mathcal{C}_A$ which contains Γ, $\Gamma, x \colon A$ and its projection. The process consists of the construction of a sequence $\{G_i\}_i$ of finite subcategories of $\mathcal{C}_\Gamma$ such that $G_0 = G$, and that G_{n+1} is computed by the process described below from G_n, and where $G_n \subseteq G_{n+1}$ for all $n \geq 0$. We say that an object or morphism of $\mathcal{C}_\mathcal{A}$ is *constructible by expansion* if there exists an $n \geq 0$ such that the given object or morphism is contained in G_n. The process of constructing G_{n+1} from G_n consists of applying the following rules to each context $\Delta = \Gamma, y_1 \colon B_1, \ldots, y_m \colon B_m$ of G_n:

E1 For each sort symbol S with a well-formed introductory rule

$$z_1 \colon C_1, \ldots, z_n \colon C_m \vdash A(z_1, \ldots, z_w) \text{ type}$$

such that G_n contains sections

$$\begin{gathered}
[\langle y_1, \ldots, y_m, t_1 \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_1 \colon C_1] \\
[\langle y_1, \ldots, y_m, t_2 \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_2[z_1 \leftarrow t_1]] \\
\vdots \\
[\langle y_1, \ldots, y_m, t_w \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_w[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}]]
\end{gathered}$$

then we add the context $[\Delta, y \colon A(t_1, \ldots, t_w)]$ and its projection to G_{n+1}.

E2 Add the morphisms

$$[\langle y_1, \ldots, y_m, x_i \rangle_\Gamma] \colon [\Delta] \to [\Delta, x \colon A_i]$$

$$[\langle y_1, \ldots, y_m, y_j \rangle_\Gamma] \colon [\Delta] \to [\Delta, y \colon B_j]$$

and their corresponding projections for each $1 \leq i \leq n$ and each $1 \leq j \leq m$.

E3 For every operator symbol F with a well-formed introductory rule

$$z_1 \colon C_1, \ldots, z_w \colon C_w \vdash F(z_1, \ldots, z_w) \colon C$$

if the following sections exist in G_n:

$$\begin{gathered}
[\langle y_1, \ldots, y_m, t_1 \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_1 \colon C_1] \\
[\langle y_1, \ldots, y_m, t_2 \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_2 \colon C_2[z_1 \leftarrow t_1]] \\
\vdots \\
[\langle y_1, \ldots, y_m, t_w \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_w \colon C_w[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}]]
\end{gathered}$$

then we add the section

$$[\langle y_1, \ldots, y_m, F(t_1, \ldots, t_w) \rangle_\Gamma] \colon [\Delta] \to [\Delta, z \colon C[z_1 \leftarrow t_1, \ldots, z_w \leftarrow t_w]]$$

E4 For every morphism

$$[\langle t_1, \ldots, t_{m-1} \rangle_\Gamma] \colon [\Gamma, z_1 \colon C_1, \ldots, z_w \colon C_w] \to [\Gamma, y_1 \colon B_1, \ldots, y_{m-1} \colon B_{m-1}]$$

we construct the corresponding pullback in G_{n+1}:

$$\begin{array}{ccc}
\begin{array}{c}[\Gamma, z_1 : C_1, \ldots, z_w : C_w, \\ y_m : B_m[t_1 \leftarrow y_1, \ldots, t_{m-1} \leftarrow y_{m-1}]]\end{array} & \xrightarrow{[\langle t_1, \ldots, t_{m-1}, y_m \rangle_\Gamma]} & [\Gamma, y_1 \colon B_1, \ldots, y_m \colon B_m] \\
\downarrow & & \downarrow \\
[\Gamma, z_1 \colon C_1, \ldots, z_w \colon C_w] & \xrightarrow{[\langle t_1, \ldots, t_{m-1} \rangle_\Gamma]} & [\Gamma, y_1 \colon B_1, \ldots, y_{m-1} \colon B_{m-1}]
\end{array}$$

Finally, we complete G_{n+1} by adding the compositions of all of its morphisms. When we say we add a morphism, it is implied that we also add both endpoints, and when adding an object, we are also adding its projection.

Now we will prove that every object and every morphism of $\mathcal{C}_A$ are constructible by expansion. Note that as projections are added together with the object, hence by asserting the existence of an object we are implying the existence of its projection. But to do so, we will introduce the concept of depth of an expression in order to prove a the result by induction. This definition is inspired to the one in the next chapter taken from [21].

Definition 1.17. By structural induction of expressions we define a function *depth* mapping the set of expressions and premisses to ω:

1. if x is a variable, then $\mathrm{depth}[x] = 1$;
2. if A is a sort symbol, and $t_1, \ldots, t_n$ are expressions, then
$$\mathrm{depth}[A(t_1, \ldots, t_n)] = 1 + \sum_{i=1}^{n} \mathrm{depth}[t_i];$$
3. if F is an operator symbol, and $t_1, \ldots, t_n$ are expressions, then
$$\mathrm{depth}[F(t_1, \ldots, t_n)] = 1 + \sum_{i=1}^{n} \mathrm{depth}[t_i];$$
4. if $\Gamma \equiv x_1 \colon A_1, \ldots, x_n \colon A_n$ is a premise, then
$$\mathrm{depth}[\Gamma] = \sum_{i=1}^{n} \mathrm{depth}[A_i].$$

Lemma 1.18. *Every object and every section of $\mathcal{C}_\Gamma$ are constructible by expansion.*

Proof. We perform this proof by induction over the degree of the objects and sections, where

1. if Δ is a context over Γ, then
$$\mathrm{degree}[\Delta] = \mathrm{depth}[\Delta] - \mathrm{depth}[\Gamma];$$
2. if $\Delta = \Gamma, y_1 \colon B_1, \ldots, y_m \colon B_m$ is a context and $[\langle y_1, \ldots, y_m, t\rangle_\Gamma]$ is a section of $[\Delta, x \colon A]$, then
$$\mathrm{degree}[\langle y_1, \ldots, y_m, t\rangle_\Gamma] = \mathrm{degree}[\Delta] + \mathrm{depth}[t].$$

Together with a secondary induction on derivation length.

Consider then a context $\Delta = \Gamma, y_1 \colon B_1, \ldots, y_m \colon B_m$. We want to see that $[\Delta]$ is constructible by expansion. As Δ is a context, we have that

$$\Delta' \vdash B_m \text{ type}$$

is a derived rule of U, where we denote $\Delta' = \Gamma, y_1 \colon B_1, \ldots, y_{m-1} \colon B_{m-1}$. This implies that Δ' is itself a context, and by induction hypothesis, as

$$\text{degree}[\Delta] = \text{degree}[\Delta'] + \text{depth}[B_m] \geq \text{degree}[\Delta'] + 1 > \text{degree}[\Delta'],$$

we have that $[\Delta']$ is constructible by expansion. Moreover, as the rule $\Delta' \vdash B_m$ type can only be derived by CF2(A), there exists a sort symbol S with a well-formed introductory rule

$$z_1 \colon C_1, \ldots, z_w \colon C_w \vdash A(z_1, \ldots, z_w) \text{ type}$$

and derived rules

$$\begin{aligned} &\Delta' \vdash t_1 \colon C_1 \\ &\Delta' \vdash t_2 \colon C_2[z_1 \leftarrow t_1] \\ &\qquad\vdots \\ &\Delta' \vdash t_w \colon C_w[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}] \end{aligned}$$

such that $B_m = A(t_1, \ldots, t_m)$ as expressions. This means that these are sections:

$$\begin{aligned} &[\langle y_1, \ldots, y_m, t_1 \rangle_\Gamma] \colon [\Delta'] \to [\Delta', z_1 \colon C_1] \\ &[\langle y_1, \ldots, y_m, t_2 \rangle_\Gamma] \colon [\Delta'] \to [\Delta', z_2 \colon C_2[z_1 \leftarrow t_1]] \\ &\qquad\vdots \\ &[\langle y_1, \ldots, y_m, t_w \rangle_\Gamma] \colon [\Delta'] \to [\Delta', z_w \colon C_w[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}]] \end{aligned}$$

And clearly, as

$$\begin{aligned} \text{degree}[\Delta] &= \text{degree}[\Delta'] + \text{depth}[A(t_1, \ldots, t_m)] > \\ &> \text{degree}[\Delta'] + \text{depth}[t_j] = \text{degree}[\langle y_1, \ldots, y_m \rangle_\Gamma] \end{aligned}$$

for every j, with $1 \leq j \leq w$, they are all constructible by expansion by induction hypothesis. Finally, by applying the expansion rule E1, we have that $[\Delta]$ is constructible by expansion.

Now, consider a section $[\langle y_1, \ldots, y_m, t \rangle_\Gamma]$ of $[\Delta, x \colon A]$. By definition, the rule

$$\Delta \vdash t \colon A \tag{1.1}$$

is a derived rule of U. We have that $\text{degree}[\langle y_1, \ldots, y_m, t \rangle_\Gamma] = \text{degree}[\Delta] + \text{depth}[t]$, hence by induction hypothesis $[\Delta]$ is constructible by expansion. We now study how the rule (1.1) was derived. As it is a $\in$-rule, it must have been derived by either T1, CF1 or CF2(A). We treat each case separately:

T1 Then, there exists an expression A' such that both $\Delta \vdash t\colon A'$ and $\Delta \vdash A = A'$ are derived rules of U. The latter rule gives us that $[\Delta, x\colon A] = [\Delta, x\colon A']$, hence we have to see that the section $[\langle y_1, \ldots, y_m, t\rangle_\Gamma]$ of $[\Delta, x\colon A']$ is constructible by expansion. As the length of the derivation of $\Delta \vdash t\colon A'$ is at least one T1 application shorter that the derivation length of $\Delta \vdash t\colon A$, by secondary induction, we have that the section is constructible by expansion.

CF1 Then, there exists an i, with $1 \leq i \leq n$, such that $t = x_i$ and $A = A_i$; or there exists a j, with $1 \leq j \leq m$, such that $t = y_j$ and $A = B_j$. We just prove the first case as the other proof is equivalent. So as $[\Delta]$ is constructible by expansion, by applying the expansion rule E2 to it, we get that the section

$$[\langle y_1, \ldots, y_m, x_i\rangle_\Gamma]\colon [\Delta] \to [\Delta, x\colon A]$$

is constructible by expansion.

CF2(A) Then, there is an operator symbol F, with a well-formed introductory rule

$$z_1\colon C_1, \ldots, z_w\colon C_w \vdash F(z_1, \ldots, z_w)\colon B$$

and derived rules:

$$\begin{aligned}
&\Delta \vdash t_1\colon C_1\\
&\Delta \vdash t_2\colon C_2[z_1 \leftarrow t_1]\\
&\qquad\vdots\\
&\Delta \vdash t_w\colon C_w[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}]
\end{aligned}$$

such that $t = F(t_1, \ldots, t_w)$ and $A = B[z_1 \leftarrow t_1, \ldots, z_w \leftarrow t_w]$. Then, as

$$\begin{aligned}
\mathrm{degree}[\langle y_1, \ldots, y_m, F(t_1, \ldots, t_m)\rangle_\Gamma] &= \mathrm{degree}[\Delta] + \mathrm{depth}[F(t_1, \ldots, t_w)] >\\
&> \mathrm{degree}[\Delta] + \mathrm{depth}[t_j] = \mathrm{degree}[\langle y_1, \ldots, y_m, t_j\rangle_\Gamma]
\end{aligned}$$

for all j, with $1 \leq j \leq w$, by the induction hypothesis, the following sections are constructible by expansion:

$$\begin{aligned}
&[\langle y_1, \ldots, y_m, t_1\rangle_\Gamma]\colon [\Delta'] \to [\Delta', z_1\colon C_1]\\
&[\langle y_1, \ldots, y_m, t_2\rangle_\Gamma]\colon [\Delta'] \to [\Delta', z_2\colon C_2[z_1 \leftarrow t_1]]\\
&\qquad\vdots\\
&[\langle y_1, \ldots, y_m, t_w\rangle_\Gamma]\colon [\Delta'] \to [\Delta', z_w\colon C_w[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}]]
\end{aligned}$$

Then by applying the expansion rule E3, we get that the section

$$[\langle y_1, \ldots, y_m, t\rangle_\Gamma]\colon [\Delta] \to [\Delta, x\colon A]$$

is constructible by expansion. □

Theorem 1.19. *Every object and every morphism of $\mathcal{C}_A$ are constructible by expansion.*

Proof. From the last lemma we have that both objects and sections are constructible by expansion. We now have to prove that any morphism is constructible by expansion. Let $[\langle t_1, \ldots, t_m \rangle_\Gamma] : [\Gamma, y_1 \colon B_1, \ldots, y_m \colon B_m] \to [\Gamma, z_1 \colon C_1, \ldots, z_w \colon C_w]$ be a morphism on $\mathcal{C}_\Gamma$. We will denote $\Delta \equiv \Gamma, y_1 \colon B_1, \ldots, y_m \colon B_m$. Then we know that the following rules

$$\begin{aligned} &\Delta \vdash t_1 \colon C_1 \\ &\Delta \vdash t_2 \colon C_2[z_1 \leftarrow t_1] \\ &\qquad\vdots \\ &\Delta \vdash t_w \colon C_2[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}] \end{aligned}$$

are derivable rules of U. So, by applying the lemma, we have that the following sections are constructible by expansion:

$$\begin{aligned} &[\langle y_1, \ldots, y_m, t_1 \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_1 \colon C_1] \\ &[\langle y_1, \ldots, y_m, t_2 \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_2 \colon C_2[z_1 \leftarrow z_2]] \\ &\qquad\vdots \\ &[\langle y_1, \ldots, y_m, t_w \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_w \colon C_w[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}]] \end{aligned}$$

Now we show that the morphism

$$[\langle y_1, \ldots, y_m, t_1, \ldots, y_w \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_1 \colon C_1, \ldots, z_w \colon C_w]$$

is constructible by expansion. To do this, we prove that, given j with $2 < j \leq n$, if

$$[\langle y_1, \ldots, y_m, t_1, \ldots t_{j-1} \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_1 \colon C_1, \ldots, z_{j-1} \colon C_{j-1}]$$

$$[\langle y_1, \ldots, y_m, t_j \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_j \colon C_j[z_1 \leftarrow t_1, \ldots, z_{j-1} \leftarrow t_{j-1}]]$$

are constructible by expansion, then so is

$$[\langle y_1, \ldots, y_m, t_1, \ldots, t_j \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_1 \colon C_1, \ldots, z_j \colon C_j]$$

By the previous lemma, the object $[\Delta, z_1 \colon C_1, \ldots, z_j \colon C_j]$ is constructible by expansion, and by applying the expansion rule E4 on it, the following diagram is constructible by expansion:

$$\begin{array}{ccc} [\Delta, z_j \colon C_j[t_1 \leftarrow z_1, \ldots, t_j \leftarrow z_j]] & \xrightarrow{\langle t_1, \ldots, t_{j-1}, z_j \rangle_\Delta} & [\Delta, z_1 \colon C_1, \ldots, z_j \colon C_j] \\ \downarrow & & \downarrow \\ \Delta & \xrightarrow{\langle t_1, \ldots, t_{j-1} \rangle_\Delta} & \Delta, z_1 \colon C_1, \ldots, z_{j-1} \colon C_{j-1} \end{array}$$

and by composition we get that

$$[\langle y_1, \ldots, y_m, t_1, \ldots, t_{j-1}, z_j \rangle_\Gamma] \circ [\langle y_1, \ldots, y_m, t_j \rangle_\Gamma] = [\langle y_1, \ldots, y_m, t_1, \ldots, t_j \rangle_\Gamma]$$

is constructible by expansion. It remains to see that the morphism

$$[\langle z_1, \ldots, z_w \rangle_\Gamma] \colon [\Delta, z_1 \colon C_1, \ldots, z_w \colon C_w] \to [\Gamma, z_1 \colon C_1, \ldots, z_w \colon C_w]$$

is constructible by expansion. As $\Gamma, z_1 \colon C_1, \ldots, z_w \colon C_w$ is a context, by Lemma 1.18, $[\Delta, z_1 \colon C_1, \ldots, z_j \colon C_j]$ is constructible by expansion. And by repeated application of the rule E4 gives us the following diagram:

$$\begin{array}{ccc}
[\Delta, z_1 \colon C_1, \ldots, z_w \colon C_w] & \xrightarrow{\langle z_1, \ldots, z_w \rangle_\Gamma} & [\Gamma, z_1 \colon C_1, \ldots, z_w \colon C_w] \\
\downarrow & & \downarrow \\
[\Delta, z_1 \colon C_1, \ldots, z_{w-1} \colon C_{w-1}] & \xrightarrow{\langle z_1, \ldots, z_{w-1} \rangle_\Gamma} & [\Gamma, z_1 \colon C_1, \ldots, z_{w-1} \colon C_{w-1}] \\
\downarrow & & \downarrow \\
\vdots & & \vdots \\
\downarrow & & \downarrow \\
[\Delta, z_1 \colon C_1] & \xrightarrow{\langle z_1 \rangle_\Gamma} & \Gamma, z_1 \colon C_1 \\
\downarrow & & \downarrow \\
[\Delta] & \xrightarrow{\langle \rangle_\Gamma} & \Gamma
\end{array}$$

This completes the proof of Theorem 1.19. □

Chapter 2

Calculus of Constructions

In Chapter 1, we established a pipeline for autonomous theorem proving: take a theory, translate its syntax into a Contextual Category, and reframe proof-finding as the geometric problem of finding a section in a graph.

In this chapter, we scale up the complexity to the Calculus of Constructions (CoC). Introduced by Coquand and Huet [6], CoC is a powerful higher-order typed lambda calculus. It serves as the foundation for the Calculus of Inductive Constructions, the type theory powering modern proof assistants like Rocq (formerly Coq) [6].

The primary additions in CoC, compared to Generalized Algebraic Theories (GATs), are:

1. **Product Types (Π-types):** These are first-class citizens, meaning dependent function types exist at the base level rather than just in the theory's signature.

2. **The Universe of Propositions (Prop):** A type universe whose terms are themselves types, allowing for higher-order logic and universal quantification over types.

We define syntax of CoC, map it to a richer categorical structure called Doctrine of Constructions, and extend our Expansion Algorithm to navigate this new, highly complex search space.

2.1 Syntactic Formulation

In this section, we formally define the syntax of Calculus of Constructions. We use the approach of Streicher in [21], since the original definition in [6] is characterized by its syntax overloading, making it much harder to follow. The latter just use $[x : A]B$ to present product types, functional abstraction and universal

quantification. Also, properties are presented as terms of type Prop and as types themselves. To make our notation more readable, we will be using a type constructor Proof that, given a proposition $p\colon \mathrm{Prop}$, yields $\mathrm{Proof}(p)$ the type of proofs of proposition p.

First, we define the class of *pre-well-formed type expressions*, ranged over by capital letters, and the class of *pre-well-formed object expressions*, ranged over by lower case letters, by mutual recursion using Backus-Naur form [1]:

$$
\begin{array}{rll}
E ::= & (\Pi x\colon E)\, E & \text{product of types} \\
 & \mid \mathrm{Prop} & \text{type of propositions} \\
 & \mid \mathrm{Proof}(t) & \text{type of proofs of proposition } t
\end{array}
$$

$$
\begin{array}{rll}
t ::= & x & \text{variable} \\
 & \mid (\lambda x\colon E)t & \text{functional abstraction} \\
 & \mid \mathrm{App}([x\colon E]E, t, t) & \text{function application} \\
 & \mid (\forall x\colon E)t & \text{universal quantification}
\end{array}
$$

Observation 2.1. We write $\mathrm{App}([x\colon E]E, t, t)$ in order to keep track of the types involved in the application of the two terms, and it kept explicit in order to make applying structural induction over the expressions easier. This notation is discussed further in [21].

We say a *pre-context* is a syntactical expression of the form $x_1\colon A_1, \ldots, x_n\colon A_n$, where $x_1, \ldots, x_n$ are pairwise syntactically different variables. Contexts are ranged over by capital greek letters. We define $\mathrm{Var}(\Gamma)$ as the set of variables declared in Γ. We will not distinguish expressions which are equivalent up to renaming of variables (α-conversion), to do this we will be using *deBruijn indices* They are a method to refer to declarations not by name (variables) but by numbers which tell how far one has to go back into the current context of an expression to find the declaration of the thing it refers to. We define a *pre-judgment* to be a syntactic expression of one of the following forms:

A type	$A$is a type
$A = B$	A and B are equal types
$t\colon A$	t is an object of type A
$t = s\colon A$	t and s are equal objects of type A

we will use the letter $\mathcal{J}$ to refer to pre-judgments. We define a *pre-sequent* as a syntactic expression of the following form:

Γ ok	Γ is a context
$\Gamma = \Delta$	Γ and Δ are equal contexts
$\Gamma \vdash \mathcal{J}$	the judgement $\mathcal{J}$ holds w.r.t. Γ

Finally, we define the set of *sequents*, i.e., valid pre-sequents, to be generated by the set of rules in A.2. Sequents are ranged over by capital italic letters. This set of rules is more extent that the original in order to make explicit rules for handling contexts and to add more rules stating equality of types and objects. We can distinguish different kinds of rules for different purposes:

1. Context Rules: their purpose is to make contexts first-class citizens of CoC, unlike in GATs where they were defined outside the rule system. Theses includes: context formation rules and context equality rules

2. Structural Rules for Judgments: their purpose is to form and serve as equality rules for judgments.

3. Rules for Product Types: these rules include everything needed for working with products.

4. Rules for Propositions: finally, the machinery to work with the Prop universe, Proof and $\forall$.

2.2 Doctrines of Constructions

We leverage the concept of Context Categories introduced in the earlier chapter to serve as a basic structure for handling dependent types, and add more conditions so that it can handle products of families of types, universal quantification and the type of properties, in order to develop a categorical structure that captures the interpretation of the Calculus of Constructions.

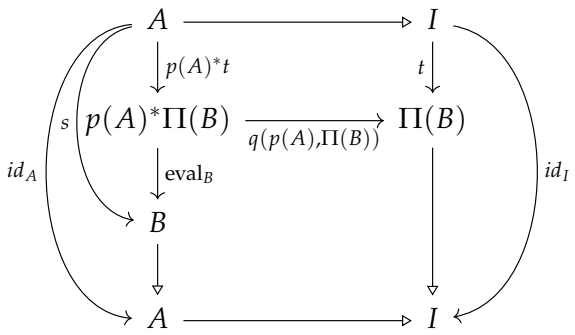
Although products might be added directly in the GAT structure, we enforce them to the base level as they are needed to the definition of universal quantification.

Definition 2.2. A *contextual category with products of families of types* is a category $\mathcal{C}$ together with two mappings Π and eval defined over the collection of objects B of $\mathcal{C}$ with $level(B) \geq 2$, such that:

1. Whenever $I \lhd A \lhd B$, then $I \lhd \Pi(B)$ and $eval_B \colon p(A)^*\Pi(B) \to B$ is a morphism over A

$$\begin{array}{ccc} p(A)^*\Pi(B) & \xrightarrow{\text{eval}_B} & B \\ & {\scriptstyle p(p(A)^*\Pi(B))} \searrow \quad \swarrow {\scriptstyle p(B)} & \\ & A & \end{array}$$

such that for all $s \in \text{Sections}(B)$ there is a unique section $t \in \text{Sections}(\Pi(B))$ with $\text{eval}_B \circ p(A)^* t = s$. This unique section t is called $\text{curry}(s)$.



Observation 2.3. as any section t of $\Pi(B)$ gives a section $eval_B \circ p(A)^* t$ of B, hence we have a canonical 1-1-correspondence of $\text{Sections}(B)$ and $\text{Sections}(\Pi(B))$.

2. For any object J and any morphism $f\colon J \to I$, the following conditions are satisfied:

$$f^*(\Pi(B)) = \Pi(f^*B) \quad \text{and} \quad f^*(eval_B) = eval_{f^*B}.$$

It follows from the definition that substitution preserves functional abstraction or currying:

Lemma 2.4. *Let $\mathcal{C}$ be a contextual category with products of families of types. Let $I \lhd A \lhd B$ be objects and $f\colon J \to I$ a morphism in $\mathcal{C}$. If t is a section of B, then*

$$f^*(curry(B)) = curry(f^*B).$$

Notice that in any contextual category with products of families of types, the following equations hold:

$$\text{eval}_B \circ p(A)^* \text{curry}(s) = s \tag{β}$$

$$\text{curry}(\text{eval}_B \circ p(A)^* t) = t \tag{η}$$

The first equation corresponds to the β-rule of typed λ-calculi and the second equation corresponds to the η-rule of typed λ-calculi. Contextual Categories with products correspond to λ-calculus with dependent types.

Definition 2.5. Let $\mathcal{C}, \mathcal{C}'$ be two contextual categories with products of families of types, we a contextual functor $F\colon \mathcal{C} \to \mathcal{C}'$ *preserves products of families of types* if for $1 \lhd A \lhd B$ in $\mathcal{C}$,

$$F\Pi(B) = \Pi(FB) \quad \text{and} \quad F\text{eval}_B = \text{eval}_{FB}$$

Now we add the type of propositions. To do it, we find objects in our category whose sections, i.e., its terms, correspond to objects.

Definition 2.6. Let $\mathcal{C}$ be a context category. An *enumeration of types of* $\mathcal{C}$ is an object T of $\mathcal{C}$ with $\text{level}(T) = 2$.

An enumeration of types T of $\mathcal{C}$ is called a *collection of types* of $\mathcal{C}$ if for all objects A and morphisms $f, g\colon A \to \text{parent}(T)$,

$$f^*T = g^*T \quad \Longrightarrow \quad f = g.$$

It is the object $\text{parent}(T)$ that we are interested in.

And finally, by wrapping everything up and adding universal quantification we have the desired structure:

Definition 2.7. A *doctrine of constructions* is a contextual category with products of families of types together with a collection of types T such that for all $A, B \in Ob\mathcal{C}$ with $A \lhd B$ and $f\colon B \to \text{Prop}$, where $\text{Prop} = \text{parent}T$, there exists a necessarily unique morphism $g\colon A \to \text{Prop}$ also denoted by $\forall(f)$ such that

$$g^*T = \forall(f)^*T = \Pi(f^*T).$$

Definition 2.8. Let $\mathcal{C}, \mathcal{C}'$ be two doctrines of constructions. A *functor of doctrines of constructions* is a contextual functor $F\colon \mathcal{C} \to \mathcal{C}'$ that preserves products of families of types such that if T and T' are the selected collections of types of $\mathcal{C}$ and $\mathcal{C}'$ respectively, $FT = T'$, and for any $A \lhd B$ in $\mathcal{C}$, and $f\colon B \to \text{Prop}$, $F(\forall(f)) = \forall(F(f))$.

We consider the category **DoC** of doctrines of constructions and functors of doctrines of constructions.

Now we see that doctrines of constructions correspond to models of Calculus of Constructions.

2.3 Interpretation of Calculus of Constructions

In this section we define the interpretation of Calculus of Constructions in arbitrary doctrines of constructions.

To do this, fixed one doctrine of constructions, an **interpretation function** $\mathcal{M}$ should be a mapping such that:

1. for every pre-context Γ of length n, if $\mathcal{M}[\Gamma]$ is defined, then $\mathcal{M}[\Gamma]$ is an object of level n;

2. for every pre-context of length n and type expression A, if $\mathcal{M}[\Gamma; A]$ is defined then $\mathcal{M}[\Gamma; A]$ is an object of level $n+1$ and $\mathcal{M}[\Gamma] \lhd \mathcal{M}[\Gamma; A]$;

3. for every pre-context Γ of length n and object expression t, if $\mathcal{M}[\Gamma; t]$ is defined then it is a morphism $s\colon \mathcal{M}[\Gamma] \to A$ where A is an object of level $n+1$ such that
$$\mathcal{M}[\Gamma] \lhd A \quad \text{and} \quad p(A) \circ s = id_{\mathcal{M}[\Gamma]}.$$

We want to build this function by induction. We need an auxiliary definition:

Definition 2.9. By structural induction of expressions we define a function **depth** mapping pre-well-formed expressions and pre-contexts to ω:
$$\text{depth}[x] = 1$$
$$\text{depth}[\text{Prop}] = 1$$
$$\text{depth}[\text{Proof}(A)] = \text{depth}[A] + 1$$
$$\text{depth}[(\Pi x\colon A)B] = \text{depth}[A] + \text{depth}[B] + 1$$
$$\text{depth}[(\lambda x\colon A)t] = \text{depth}[A] + \text{depth}[t] + 1$$
$$\text{depth}[(\forall x\colon A)t] = \text{depth}[A] + \text{depth}[t] + 1$$
$$\text{depth}[\text{App}([x\colon A]B, t, s)] = \text{depth}[A] + \text{depth}[B] + \text{depth}[t] + \text{depth}[s] + 1$$
$$\text{For } \Gamma \equiv x_1\colon A_1, \ldots, x_n\colon A_n, \quad \text{depth}[\Gamma] = \sum_{i=1}^{n} \text{depth}[A_i]$$

We define $\mathcal{M}$ by induction on the level of its arguments, where
$$\text{level}[\Gamma] = \text{depth}[\Gamma],$$
$$\text{level}[\Gamma; A] = \text{depth}[\Gamma] + \text{depth}[A],$$
$$\text{level}[\Gamma; t] = \text{depth}[\Gamma] + \text{depth}[t]$$
by the following clauses:

1. $\mathcal{M}[] = 1$;

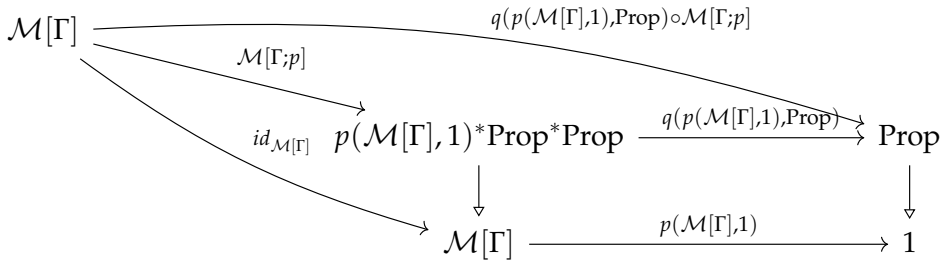
2. $\mathcal{M}[\Gamma, x\colon A] = \mathcal{M}[\Gamma; A]$;

3. $\mathcal{M}[\Gamma, (\Pi x\colon A)B] = \Pi(\mathcal{M}[\Gamma, x\colon A; B])$ if $\mathcal{M}[\Gamma, x\colon A; B]$ is defined;

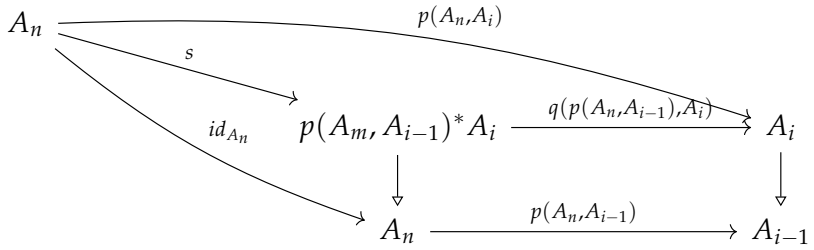
4. $\mathcal{M}[\Gamma, \text{Prop}] = p(\mathcal{M}, 1)^*\text{Prop}$ if $\mathcal{M}[\Gamma]$ is defined;

5. $\mathcal{M}[\Gamma; \text{Proof}(p)] = \mathcal{M}[\Gamma; p]^* p(\mathcal{M}[\Gamma], 1)^* T$ if $\mathcal{M}[\Gamma; p]$ is defined and a section

of $p(\mathcal{M}[\Gamma], 1)^*\mathrm{Prop}$



6. $\mathcal{M}[\Gamma, x] = s$ provided that $\Gamma \equiv x_1 \colon A_1, \ldots, x_n \colon A_n$ and $x_i \equiv x$ and $\mathcal{M}[\Gamma]$ is defined and $\mathcal{M}[\Gamma] = A_n \rhd \cdots \rhd A_1 \rhd A_0 \rhd 1$ and s as follows



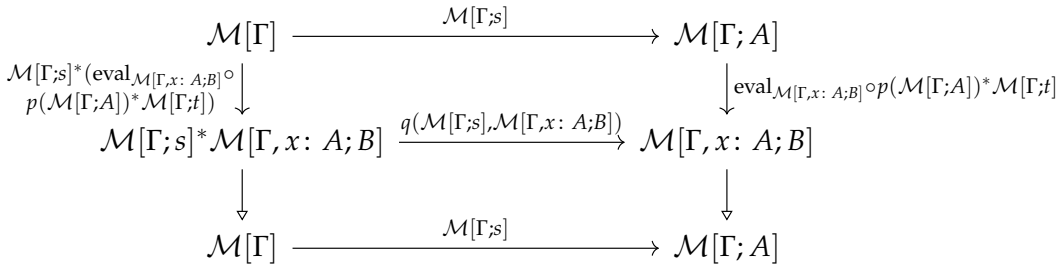
or, in other terms, $\mathcal{M}[\Gamma; x_i] = p(A_n, A_i)^*\Delta(A_i)$ where $\Delta(A_i)$ is the unique arrow $m \colon A_i \to p(A_i)^* A_i$ such that

$$p(p(A_i)^* A_i) \circ m = q(p(A_i), A_i) \circ m = id_{A_i};$$

7. $\mathcal{M}[\Gamma; (\lambda x \colon A)t] = \mathrm{curry}(\mathcal{M}[\Gamma, x \colon A; t])$ if $\mathcal{M}[\Gamma, x \colon A; t]$ is defined. By induction hypothesis we have that $\mathcal{M}[\Gamma] \lhd \mathcal{M}[\Gamma; A] \lhd \mathrm{cod}(\mathcal{M}[\gamma, x \colon A; t])$ and s is a section of $\mathcal{M}[\Gamma, x \colon A; t]$ and therefore we have:

$$\mathrm{eval}_{\mathrm{cod}(\mathcal{M}[\Gamma, x \colon A; t])} \circ p(\mathcal{M}[\Gamma; A])^* \mathcal{M}[\Gamma; (\lambda x \colon A)t] = \mathcal{M}[\Gamma, x \colon A; t];$$

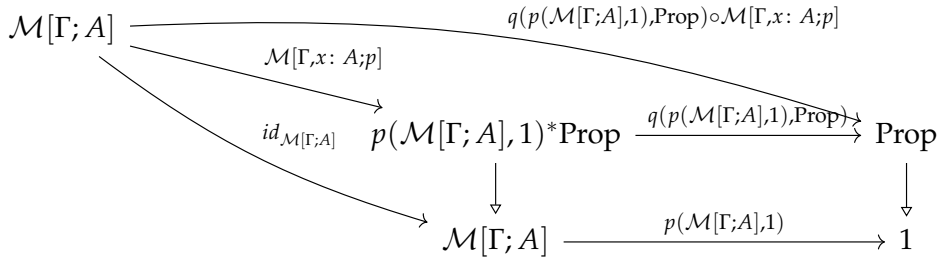
8. $\mathcal{M}[\Gamma; \mathrm{App}([x \colon A]B), t, s] = \mathcal{M}[\Gamma; s]^*(\mathrm{eval}_{\mathcal{M}[\Gamma, x \colon A; B]} \circ p(\mathcal{M}[\Gamma; A])^* \mathcal{M}[\Gamma; t])$ provided that $\mathcal{M}[\Gamma; t]$, $\mathcal{M}[\Gamma; s]$, $\mathcal{M}[\Gamma, x \colon A; B]$ are defined, and $\mathcal{M}[\Gamma; s]$ is a section of $\mathcal{M}[\Gamma; A]$ and $\mathcal{M}[\Gamma; t]$ is a section of $\Pi(\mathcal{M}[\Gamma, x \colon A; B])$



9. $\mathcal{M}[\Gamma; (\forall x \colon A)p] = s$ provided $\mathcal{M}[\Gamma, x \colon A; p]$ is a section of $p(\mathcal{M}[\Gamma; A], 1)^*\mathrm{Prop}$

and s is the unique section of $p(\mathcal{M}[\Gamma], 1)$ such that

$$\forall(q(p(\mathcal{M}[\Gamma; A], 1), \mathrm{Prop}) \circ \mathcal{M}[\Gamma, x\colon A; p]) = q(p(\mathcal{M}[\Gamma; A], 1), \mathrm{Prop}) \circ \mathcal{M}[\Gamma, x\colon A; p]$$



Theorem 2.10 (Correctness Theorem)**.** *For any doctrine of constructions the following correctness conditions, for $\mathcal{M}$ are satisfied:*

1. *if Γ ok is derivable, then $\mathcal{M}[\Gamma]$ is defined;*
2. *if $\Gamma \vdash A$ type is derivable, then $\mathcal{M}[\Gamma; A]$ is defined;*
3. *if $\Gamma \vdash t\colon A$ is derivable, then $\mathcal{M}[\Gamma; A]$ and $\mathcal{M}[\Gamma; t]$ are both defined and $\mathcal{M}[\Gamma; t]$ is a section of $\mathcal{M}[\Gamma; A]$;*
4. *if $\Gamma = \Delta$ is derivable, then $\mathcal{M}[\Gamma]$ and $\mathcal{M}[\Delta]$ are defined and $\mathcal{M}[\Gamma] = \mathcal{M}[\Delta]$;*
5. *if $\Gamma \vdash A = B$ is derivable, then $\mathcal{M}[\Gamma; A]$ and $\mathcal{M}[\Gamma; B]$ are both defined and $\mathcal{M}[\Gamma; A] = \mathcal{M}[\Gamma; B]$;*
6. *if $\Gamma \vdash t = s \in A$ is derivable, then $\mathcal{M}[\Gamma; A]$, $\mathcal{M}[\Gamma; t]$ and $\mathcal{M}[\Gamma; s]$ are defined, both $\mathcal{M}[\Gamma; t]$ and $\mathcal{M}[\Gamma; s]$ are sections of $\mathcal{M}[\Gamma; A]$ and $\mathcal{M}[\Gamma; t] = \mathcal{M}[\Gamma; s]$.*

Hence, doctrines of constructions are models of Calculus of Constructions.

2.4 The Term Model of Calculus of Constructions

In this section we construct a doctrine of constructions $\mathcal{C}_I$ as a *term model* of calculus of constructions such that the interpretation in this model interprets provably well-defined contexts, types and objects, and identifies only those contexts, types or objects which are provably equal up to renaming of the variables bound in contexts.

In more technical terms, this means that for the interpretation function $\mathcal{M}$ assigning meaning to calculus of constructions in the doctrine of constructions $\mathcal{C}_I$, the following requirements must be fulfilled:

1. for all pre-contexts Γ and Δ, if $\mathcal{M}[\Gamma] = \mathcal{M}[\Delta]$, then $\Gamma = \Delta$ is derivable;
2. for all pre-contexts Γ and Δ and pre-type-expressions A and B, if $\mathcal{M}[\Gamma; A] = \mathcal{M}[\Delta; B]$, then $\Gamma = \Delta$ and $\Gamma \vdash A = B$ are derivable;
3. for all pre-contexts Γ and Δ, pre-type-expressions A and B and pre-object-expressions t and s, if $\mathcal{M}[\Gamma; t] = \mathcal{M}[\Delta; s]$ is a section of $\mathcal{M}[\Gamma; A] = [\Delta; B]$, then $\Gamma = \Delta$, $\Gamma \vdash A = B$ and $\Gamma \vdash t = s\colon A$ are derivable.

In order to be able to identify contexts up to α-conversion, we consider a fixed numerable set of variables: $v_1, \ldots, v_n, \ldots$. We say that a context $\Gamma \equiv x_1\colon A_1, \ldots, x_n\colon A_n$ is in **α-normal form** iff $x_i \equiv v_i$ for all $1 \leq i \leq n$. Moreover, we say that a pre-context Γ is admissible iff it is in α-normal form and $\vdash \Gamma$ ok.

Now we define $\mathcal{C}_I$: The objects of $\mathcal{C}_I$ are the admissible contexts modulo provable equivalence, i.e., that we will consider two contexts Γ and Δ equal iff $\Gamma = \Delta$ is derivable. By induction over the structure of derivations, one can see that $\Gamma = \Delta$ implies that Γ and Δ have the same length as pre-contexts, so we can define the level of an object of $\mathcal{C}_I$ as the length of any of its representatives. We will write $[\Gamma]$ to represent the class containing Γ. If $\Gamma \equiv v_1\colon A_1, \ldots, v_n\colon A_n$ is an admissible context, then we define

$$\text{parent}[\Gamma] = [v_1\colon A_1, \ldots, v_{n-1}\colon A_{n-1}],$$

that is well defined according to rule CREFL1.

If $\Gamma \equiv v_1\colon A_1, \ldots, v_n\colon A_n$ and $\Delta \equiv v_1\colon B_1, \ldots, v_m\colon B_m$ are admissible contexts, a morphism from $[\Gamma]$ to $[\Delta]$ is given by a m-tuple $\langle t_1, \ldots, t_m \rangle$ of terms such that for i, $1 \leq i \leq m$,

$$\Gamma \vdash t_i\colon B_i[v_1 \leftarrow t_1, \ldots, v_{i-1} \leftarrow t_{i-1}]$$

is derivable, if $\langle s_1, \ldots, s_m \rangle$ is another m-tuple such that for i, $1 \leq i \leq m$

$$\Gamma \vdash s_i\colon B_i[v_1 \leftarrow s_1, \ldots, v_{i-1} \leftarrow s_{i-1}]$$

is derivable, then $\langle t_1, \ldots, t_m \rangle$ and $\langle s_1, \ldots, s_m \rangle$ are considered equal iff for i, $1 \leq i \leq m$

$$\Gamma \vdash t_i = s_i\colon B_i[v_1 \leftarrow s_1, \ldots, v_{i-1} \leftarrow s_{i-1}]$$

is derivable, according to rules CREFL1 and CREFL2 the definition is independent from the choice of representatives. We write $[\langle t_1, \ldots, t_m \rangle]\colon [\Gamma] \to [\Delta]$ for the class of m-tuples equatable to $\langle t_1, \ldots, t_m \rangle$. If we consider $\Theta \equiv v_1\colon C_1, \ldots, v_k\colon C_k$ is another admissible context and $[\langle s_1, \ldots, s_k \rangle]\colon [\Delta] \to [\Theta]$, then the composition of $[\langle t_1, \ldots, t_m \rangle]$ with $[\langle s_1, \ldots, s_k \rangle]$ is defined as

$$[\langle s_1, \ldots, s_k \rangle] \circ [\langle t_1, \ldots, t_m \rangle] = [\langle s_1[v_1 \leftarrow t_1, \ldots, v_m \leftarrow t_m], \ldots, s_k[v_1 \leftarrow t_1, \ldots, v_m \leftarrow t_m] \rangle].$$

If $\Gamma \equiv v_1\colon A_1, \ldots, v_n\colon A_n$ is an admissible context with $n \geq 1$ then the *canonical*

projection map $p([\Gamma])\colon [\Gamma] \to \text{parent}[\Gamma]$ is represented by the tuple $\langle v_1, \ldots, v_{n-1}\rangle$. Let $\Gamma \equiv v_1\colon A_1, \ldots, v_n\colon A_n$ and $\Delta \equiv v_1\colon B_1, \ldots, v_m\colon B_m$ admissible contexts and $t = [\langle t_1, \ldots, t_m\rangle]\colon [\Gamma] \to [\Delta]$ and C a pre-well-formed type expression with $\Delta \vdash C$ type, i.e. $\Delta, v_{m+1}\colon C$ derivable is an admissible context, then

$$t^*[\Delta, v_{m+1}\colon C] = [\Delta, v_{n+1}\colon C[v_1 \leftarrow t_1, \ldots, v_m \leftarrow t_m]]$$

$$q([t], [\Delta, v_{m+1}\colon C]) = [\langle t_1, \ldots, t_m, v_{n+1}\rangle]\colon t^*[\Delta, v_{m+1}\colon C] \to [\Delta, v_{m+1}\colon C].$$

If $\Gamma \equiv v_1\colon A_1, \ldots, v_n\colon A_n, v_{n+1}\colon B, v_{n+2}\colon C$ is an admissible context we define

$$\Pi([\Gamma]) = [\Gamma, v_{n+1}\colon (\Pi v_{n+1}\colon B)C]$$

$$ev_{[\Gamma]} = [\langle v_1, \ldots, v_n, v_{n+1}, \text{App}([v_{n+1}\colon B]C, v_{n+2}, v_{n+1})\rangle].$$

which can be seen to be a morphism from $[\Gamma, v_{n+1}\colon B, v_{n+2}\colon (\Pi v_{n+1}B)C]$ to $[\Gamma]$ over the object $[v_1\colon A_1, \ldots, v_n\colon A_n]$. The object Prop of C_I is defined as $[v_1\colon \text{Prop}]$ and the object T of C_I is defined as the object $[v_1\colon \text{Prop}, v_2\colon \text{Proof}(v_1)]$. If $\Gamma \equiv v_1\colon A_1, \ldots, v_n\colon A_n, v_{n+1}\colon B$ and $[\langle p\rangle]$ is a morphism from $[\Gamma]$ to $[v_1\colon \text{Prop}]$ then we define

$$\forall([p]) = [\langle(\forall v_{n+1}\colon B)p\rangle]\colon [v_1\colon A_1, \ldots, v_n\colon A_n] \to [v_1\colon \text{Prop}].$$

It is proven in [21] that C_I is in fact a doctrine of constructions. Also, it is immediate to see that C_I does not interpret or identify more objects or terms that those that are provably defined as equal.

Theorem 2.11 (Completeness Theorem)**.** *Let $\mathcal{M}$ be the interpretation of calculus of construction in C_I. Then*

1. *if $\mathcal{M}[\Gamma]$ is defined then Γ ok is derivable;*
2. *if $\mathcal{M}[\Gamma; A]$ is defined then $\Gamma \vdash A$ type is derivable;*
3. *if $\mathcal{M}[\Gamma; t]$ is defined then $\Gamma \vdash t\colon A$ is derivable for some pre-type-expression A;*
4. *if $\Gamma \equiv x_1\colon A_1, \ldots, x_n\colon A_n$, $\Delta \equiv y_1\colon B_1, \ldots, y_m\colon B_m$ and $\mathcal{M}[\Gamma; A] = \mathcal{M}[\Delta; B]$, then $n = m$ and*

$$v_1\colon A_1[x_1 \leftarrow v_1, \ldots, x_n \leftarrow v_n], \ldots, v_n\colon A_n[x_1 \leftarrow v_1, \ldots, x_n \leftarrow v_n] = \\ = v_1\colon B_1[y_1 \leftarrow v_1, \ldots, y_m \leftarrow v_m], \ldots, v_m\colon B_m[y_1 \leftarrow v_1, \ldots, y_m \leftarrow v_m]$$

$$v_1\colon A_1[x_1 \leftarrow v_1, \ldots, x_n \leftarrow v_n], \ldots, v_n\colon A_n[x_1 \leftarrow v_1, \ldots, x_n \leftarrow v_n] \\ \vdash A[x_1 \leftarrow v_1, \ldots, x_n \leftarrow v_n] = B[y_1 \leftarrow v_1, \ldots, y_m \leftarrow v_m]$$

are derivable;

5. *if* $\Gamma \equiv x_1 : A_1, \ldots, x_n : A_n$, $\Delta \equiv y_1 : B_1, \ldots, y_m : B_m$ *and* $\mathcal{M}[\Gamma; t] = \mathcal{M}[\Delta, s]$, *then* $n = m$ *and*

$$v_1 : A_1[x_1 \leftarrow v_1, \ldots, x_n \leftarrow v_n], \ldots, v_n : A_n[x_1 \leftarrow v_1, \ldots, x_n \leftarrow v_n] = \\ = v_1 : B_1[y_1 \leftarrow v_1, \ldots, y_m \leftarrow v_m], \ldots, v_m : B_m[y_1 \leftarrow v_1, \ldots, y_m \leftarrow v_m]$$

is derivable, and, for some pre-type-expression A,

$$v_1 : A_1[x_1 \leftarrow v_1, \ldots, x_n \leftarrow v_n], \ldots, v_n : A_n[x_1 \leftarrow v_1, \ldots, x_n \leftarrow v_n] \\ \vdash t[x_1 \leftarrow v_1, \ldots, x_n \leftarrow v_n] = s[y_1 \leftarrow v_1, \ldots, y_m \leftarrow v_m] : A$$

is derivable.

It can be also shown that $\mathcal{C}_I$ is the initial object of the category of doctrines of constructions and structure preserving functors, named **DoC**. Hence tha term model satisfies the properties of soundness, completeness and initiality that we wanted.

2.5 Extending Calculus of Constructions and its Interpretation

In this section we show how to add symbols to Calculus of Constructions, the same thing that we could do in GATs. We need to understand the current CoC as just as a minimal starting point, same as the empty theory for GATs. Inspired by the constructions for Categories with Families presented in [2], we inductively define the concept of signature Σ (corresponding to theory when talking about GATs), CoC extended by Σ, the corresponding category $\mathbf{DoC}_\Sigma$ and interpretation functions $\mathcal{M}^\Sigma$.

As a starting case, we consider the empty signature $\Sigma = \emptyset$, together with $\mathrm{CoC}_\emptyset$ representing the current syntax of Calculus of Constructions. The category $\mathbf{DoC}_\emptyset$ is exactly **DoC**, and the notion of interpretation morphism stays the same.

Now, for the inductive step, assume we have defined Σ as a valid signature, and the associated syntax CoC_Σ, category $\mathbf{DoC}_\Sigma$ and interpretation function $\mathcal{M}$. Then

- **Adding a sort symbol:** Let Γ be a context in CoC_Σ. We can extend Σ with a sort symbol S relative to Γ, to obtain the signature $\Sigma' = (\Sigma, (\Gamma, S))$. Then, we extend CoC_Σ to obtain $\mathrm{CoC}_{\Sigma'}$ by adding a production of pre-type-expressions $E ::= S$ and the inference rule $\Gamma \vdash S$. The category $\mathbf{DoC}_{\Sigma'}$ has as objects pairs $(\mathcal{C}, S_\mathcal{C})$, where $\mathcal{C}$ is an object of $\mathbf{DoC}_\Sigma$ and $\mathcal{M}_\mathcal{C}[\Gamma] \lhd A_\mathcal{C}$. Finally, we define $\mathcal{M}'_\mathcal{C}$, for each object $\mathcal{C}$ of $\mathbf{DoC}_{\Sigma'}$, by adding $\mathcal{M}'_\mathcal{C}[\Gamma; S] = S_\mathcal{C}$.

- **Adding an operator symbol:** Let Γ be a context and A be a type under Γ of CoC_Σ, we can extend Σ with a new operator symbol f relative to Γ and A obtaining $\Sigma' = (\Sigma, (\Gamma, A, f))$. Then, we extend CoC_Σ to obtain $\mathrm{CoC}_{\Sigma'}$ by adding a production of pre-object-expressions $t ::= f$ and the inference rule $\Gamma \vdash f : A$. The category $\mathbf{DoC}_{\Sigma'}$ has as objects pairs $(\mathcal{C}, f_\mathcal{C})$, where $\mathcal{C}$ is an object of $\mathbf{DoC}_\Sigma$ and $f_\mathcal{C}$ is a section of $\mathcal{M}_\mathcal{C}[\Gamma, x : A]$. Finally, we define $\mathcal{M}'_\mathcal{C}$, for each object $\mathcal{C}$ of $\mathbf{DoC}_{\Sigma'}$, by adding $\mathcal{M}'_\mathcal{C}[\Gamma; f] = f_\mathcal{C}$.

- **Adding an equality of types:** Let Γ be a context, A, A' types under Γ of CoC_Σ, we can extend Σ with a new equation $A = A'$ relative to Γ, to obtain $\Sigma' = (\Sigma, (\Gamma, A, A'))$. Then, we extend CoC_Σ to obtain $\mathrm{CoC}_{\Sigma'}$ by adding the inference rule $\Gamma \vdash A = A'$. The category $\mathbf{DoC}_{\Sigma'}$ is a full subcategory of $\mathbf{DoC}_\Sigma$ containing the objects $\mathcal{C}$ of $\mathbf{DoC}_\Sigma$ such that $\mathcal{M}_\mathcal{C}[\Gamma; A] = \mathcal{M}_\mathcal{C}[\Gamma; A']$. $\mathcal{M}'$ is the restriction of $\mathcal{M}$ to the objects of $\mathbf{DoC}_{\Sigma'}$.

- **Adding an equality of terms:** Let Γ be a context, A a type under Γ and a, a' terms of type A under Γ of CoC_Σ. We can extend Σ with a new equation $a = a'$ relative to Γ and A, to obtain $\Sigma' = (\Sigma, (\Gamma, A, a, a'))$. Then, we extend CoC_Σ to obtain $\mathrm{CoC}_{\Sigma'}$ by adding the inference rule $\Gamma \vdash a = a' : A$. The category $\mathbf{DoC}_{\Sigma'}$ is a full subcategory of $\mathbf{DoC}_\Sigma$ containing the objects $\mathcal{C}$ of $\mathbf{DoC}_\Sigma$ such that $\mathcal{M}_\mathcal{C}[\Gamma; a] = \mathcal{M}_\mathcal{C}[\Gamma; a']$. $\mathcal{M}'$ is the restriction of $\mathcal{M}$ to the objects of $\mathbf{DoC}_{\Sigma'}$.

Under this construction, we can consider $\mathcal{C}_\Sigma$ to be the exact same construction as $\mathcal{C}_I$, but with CoC_Σ instead of $\mathrm{CoC}_\emptyset$. Following a similar proof that the original proof for $\mathcal{C}_I$ in [21] we can prove the corresponding *Completeness Theorems* and that $\mathcal{C}_\Sigma$ is the initial object of $\mathbf{DoC}_\Sigma$.

2.6 Expansion Algorithm for CoC

First, we want to redefine our problem from the previous definition for GATs to our new syntax, and rephrase it to similar categorical terms. Then we will propose an extended version of the Expansion Algorithm.

The problem now can be formalized as follows: Given a valid signature of Calculus of Constructions Σ and a derivable sequent $\Gamma \vdash A$ type in CoC_Σ, we want to find a pre-object-expression t such that $\Gamma \vdash t : A$ is a derivable rule in CoC_Σ.

We can consider this from the point of view of the category $\mathcal{C}_\Sigma$ introduced in the previous section. This means that our problem is equivalent to finding a section s of $\mathcal{M}[\Gamma; A] = [\Gamma, v_{n+1} : A]$ in $\mathcal{C}_\Sigma$, assuming Γ is a context of length n in normal form.

Moreover, we want to use the reduction of our search space we did on the previous case by considering the category over a base object. To do it we need the following result:

Definition 2.12. Given a doctrine of constructions $\mathcal{C}$, and an arbitrary object A of $\mathcal{C}$. The *relativization* of $\mathcal{C}$ to A, noted as $\mathcal{C}_A$, is defined as follows:

1. $\mathcal{C}_A$ is a category whose objects are the collection of objects B over A in $\mathcal{C}$, i.e. objects B in $\mathcal{C}$ such that $A \leq B$;

2. for two objects B, C in $\mathcal{C}_A$, a morphism from B to C over A, i.e., a morphism in $\mathcal{C}_A$, is a morphism $f : B \to C$ in $\mathcal{C}$ such that $p(C, A) \circ f = p(B, A)$, composition of morphisms is inherited from $\mathcal{C}$;

3. $\text{parent}_A(B) = \text{parent}(B)$ for $B > A$ and $\text{parent}_A(A) = \text{parent}(A)$;

4. for objects $B > A$, we define $p_A(B) = p(B) : B \to \text{parent}_A(B)$;

5. for B, C, D objects of $\mathcal{C}$ with $\text{parent}_A(D) = C$ and morphism $f : B \to C$ in $\mathcal{C}_A$ we define
$$f^{*_A} D = f^* D \quad \text{and} \quad q_A(f, D) = q(f, D);$$

6. for B in $\mathcal{C}$ with $\text{level}_A(B) \geq 2$,
$$\Pi_A(B) = \Pi(B) \quad \text{and} \quad \text{eval}_B^A = \text{eval}_B;$$

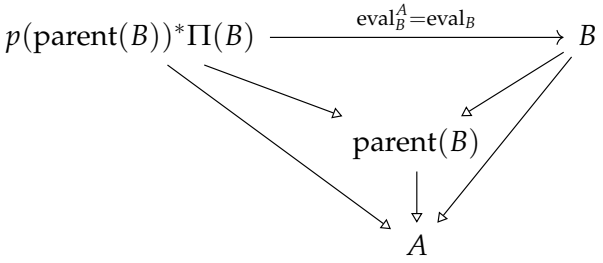
7. if T is the collection of types of $\mathcal{C}$, then
$$T_A = p(A)^* T$$
is the collection of types of $\mathcal{C}_A$.

Theorem 2.13. *The relativization $\mathcal{C}_A$ of a doctrine of constructions $\mathcal{C}$ to an object A is a doctrine of constructions.*

Proof. We already know from the first chapter that $\mathcal{C}_A$ is a contextual category. Let us check if it is also a doctrine of constructions with these extra structure we introduce.

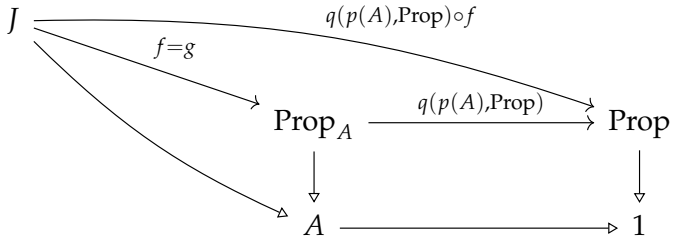
It is clear that $\mathcal{C}_A$ is a contextual category with products of families of types as for any object B of $\mathcal{C}_A$ with $\text{level}_A(B) \geq 2$, we have that $\text{level}(B) \geq 2$ and as $\mathcal{C}$ is a contextual category with products of families of types, $\Pi_A(B) = \Pi(B)$ is defined and $\text{parent}(\Pi(B)) = \text{parent}^2(B)$, and hence $\Pi_A(B) \rhd A$, hence it is well

defined. Then for $\text{eval}_B^A = \text{eval}_B \colon p(\text{parent}(B))^*\Pi(B) \to B$ is a morphism in $\mathcal{C}_A$ as by definition the following diagram commutes:



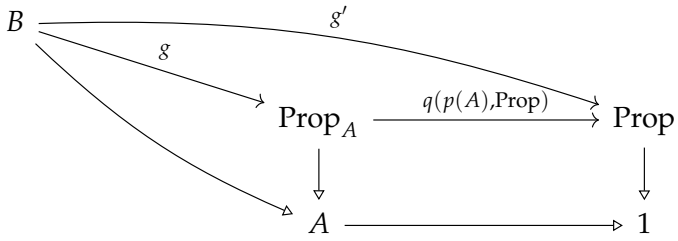
Now, if s is a section of B in $\mathcal{C}_A$, it is a section of B in $\mathcal{C}$. Hence there exists a unique section t of $\Pi(B)$ in $\mathcal{C}$ such that $eval_B \circ p(A)^*t = s$, as $p(\Pi(B)) \circ t = id_{\text{parent}(\Pi(B))}$, it is clear that t is a section of $\Pi_A(B)$ in $\mathcal{C}_A$. Now if J is an object and $f \colon J \to \text{parent}^2(B)$ is a morphism in $\mathcal{C}_A$, in particular, they belong to $\mathcal{C}$. So we have that $\Pi(f^*B) = f^*\Pi(B)$ and $\text{eval}_{f^*B} = f^*\text{eval}_B$, but by definition of Π_A and eval_A we have that $\Pi_A(f^*B) = f^*\Pi_A(B)$ and $\text{eval}_{f^*B}^A = f^*\text{eval}_B^A$. With this we conclude that $\mathcal{C}_A$ is a contextual category with products of families of types.

Now we see that $T_A = p(A)^*T$ is indeed a collection of types of $\mathcal{C}_A$. Consider then two morphisms $f, g \colon J \to \text{parent}(T_A)$, for some object A in $\mathcal{C}_A$, such that $f^*T_A = g^*T_A$. We want to see that indeed $f = g$. We have that $f^*(q(p(A), \text{Prop})^*T) = g^*(q(p(A), \text{Prop})^*T)$ and we derive that $(q(p(A), \text{Prop}) \circ f)^*T = (q(p(A), \text{Prop}) \circ g)^*T$. As T is a collection of types in $\mathcal{C}$, $q(p(A), \text{Prop}) \circ f = q(p(A), \text{Prop}) \circ g$. And finally $f = g$ as both make the following pullback diagram commute:



Finally, let B, C be objects of $\mathcal{C}_A$ and $f \colon C \to \text{Prop}_A$ a morphism over A. Then we want to see that there exists a necessarily unique morphism $g \colon B \to \text{Prop}_A$ such that $g^*T_A = \Pi_A(f^*T_A)$. As $\mathcal{C}$ holds this property, we consider $q(p(A), \text{Prop}) \circ f \colon C \to \text{Prop}$, hence there exists a necessarily unique $g' \colon B \to \text{Prop}$ such that $g'^*T = \Pi((q(p(A), \text{Prop}) \circ f)^*T) = \Pi(f^*T_A)$, and by considering g the unique

function that makes the following pullback diagram commute:



we then have that $g' = q(p(A), \mathrm{Prop}) \circ g$, hence:

$$\begin{aligned} g^* T_A &= g^*(q(p(A), \mathrm{Prop})^* T) \\ &= (q(p(A), \mathrm{Prop}) \circ g)^* T \\ &= g'^* T \\ &= \Pi((q(p(A), \mathrm{Prop}) \circ f)^* T) \\ &= \Pi(f^*(q(p(A), \mathrm{Prop})^* T)) = \Pi_A(f^* T_A) \end{aligned}$$

This concludes the proof as g is a morphism over A. □

With this result, and with the notation of Γ subscript introduced in the second chapter, we reduce our problem to finding a section of $[\Gamma, x\colon A]$ in $\mathcal{C}_{\Sigma,\Gamma}$, the relativization of $\mathcal{C}_\Sigma$ to Γ.

Now we extend the expansion algorithm from the previous chapter. We reintroduce the rules in order to match the syntax we are using for Calculus of Constructions. First, consider G to be a finite subcategory of $\mathcal{C}_{\Sigma,\Gamma}$ containing $[\Gamma]$, $[\Gamma, p\colon \mathrm{Prop}]$, $\Gamma, x\colon A$ and their projections. Now we define the expansion rules. If Δ is a context, then:

E1 For each sort symbol in the signature Σ, with an introductory rule $\Omega \vdash S$ type, such that $\Delta \equiv \Gamma, y_1\colon B_1, \ldots, y_m\colon B_m$, $\Omega \equiv z_1\colon C_1, \ldots, z_w\colon C_w$ and G_n contains sections

$$\begin{aligned} &[\Delta] \xrightarrow{\langle y_1,\ldots,y_m,t_1\rangle_\Gamma} [\Delta, z_1\colon C_1] \\ &[\Delta] \xrightarrow{\langle y_1,\ldots,y_m,t_2\rangle_\Gamma} [\Delta, z_2\colon C_2[z_1 \leftarrow t_1]] \\ &\quad\vdots \\ &[\Delta] \xrightarrow{\langle y_1,\ldots,y_m,t_w\rangle_\Gamma} [\Delta, z_w\colon C_w[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}]]. \end{aligned}$$

Then we add $[\Delta, y\colon A[z_1 \leftarrow t_1, \ldots, z_w \leftarrow t_w]]$ and its projection to G_{n+1}.

E2 If $\Delta \equiv \Gamma, y_1\colon B_1, \ldots, y_m\colon B_m$, then we add the sections

$$[\Delta] \xrightarrow{\langle y_1,\ldots,y_m,y_i\rangle_\Gamma} [\Delta, y\colon B_i]$$

$$[\Delta] \xrightarrow{\langle y_1,\ldots,y_m,x_j\rangle_\Gamma} [\Delta, x\colon A_j]$$

and their corresponding projections for each $1 \leq i \leq m$ and each $1 \leq j \leq n$.

E3 For each sort symbol in the signature Σ, with an introductory rule $\Omega \vdash F\colon A$, such that $\Delta \equiv \Gamma, y_1\colon B_1, \ldots, y_m\colon B_m$, $\Omega \equiv z_1\colon C_1, \ldots, z_w\colon C_w$ and the following sections exist in G_n

$$\begin{aligned}
&[\Delta] \xrightarrow{\langle y_1,\ldots,y_m,t_1\rangle_\Gamma} [\Delta, z_1\colon C_1]\\
&[\Delta] \xrightarrow{\langle y_1,\ldots,y_m,t_2\rangle_\Gamma} [\Delta, z_2\colon C_2[z_1 \leftarrow t_1]]\\
&\quad\vdots\\
&[\Delta] \xrightarrow{\langle y_1,\ldots,y_m,t_w\rangle_\Gamma} [\Delta, z_w\colon C_w[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}]]
\end{aligned}$$

we add the section

$$[\Delta] \xrightarrow{\langle y_1,\ldots,y_m,F[z_1\leftarrow t_1,\ldots,z_w\leftarrow t_w]\rangle_\Gamma} [\Delta, z\colon A[z_1 \leftarrow t_1, \ldots, z_w \leftarrow t_w]].$$

E4 If $\Delta \equiv \Gamma, y_1\colon B_1, \ldots, y_m\colon B_m$, for every morphism

$$[\langle t_1, \ldots, t_{m-1}\rangle_\Gamma]\colon [\Gamma, z_1\colon C_1, \ldots, z_w\colon C_w] \to [\Gamma y_1\colon B_1, \ldots, y_{m-1}\colon B_{m-1}]$$

we construct the corresponding pullback:

$$\begin{array}{ccc}
\Gamma, z_1\colon C_1, \ldots, z_w\colon C_w, y_m\colon B_m[y_1 \leftarrow t_1, \ldots, y_{m-1} \leftarrow t_{m-1}] & \xrightarrow{\langle t_1,\ldots,t_{m-1},y_m\rangle_\Gamma} & \Gamma, y_1\colon B_1, \ldots, y_m\colon B_m \\
\downarrow & & \downarrow \\
\Gamma, z_1\colon C_1, \ldots, z_w\colon C_w & \xrightarrow{\langle t_1,\ldots,t_{m-1}\rangle_\Gamma} & \Gamma, y_1\colon B_1, \ldots, y_{m-1}\colon B_{m-1}
\end{array}$$

to G_{n+1}.

E5 If $\Delta \equiv \Gamma, v_1\colon B_1, \ldots, v_m\colon B_m, v_{m+1}\colon C, v_{n+2}\colon D$ we add

$$\Pi([\Delta]) = [\Gamma, v_1\colon B_1, \ldots, v_m\colon B_m, v_{m+1}\colon (\Pi v_{m+1}\colon C)D]$$

$$\mathrm{eval}_{[\Delta]} = [\langle v_1, \ldots, v_{m+1}, \mathrm{App}([v_{m+1}\colon C]D, v_{m+2}, v_{m+1})\rangle_\Gamma].$$

a morphism from $[\Gamma, v_1\colon A_1, \ldots, v_m\colon B_m, v_{n+1}\colon C, v_{n+2}\colon (\Pi v_{n+1}\colon B)C]$ to $[\Delta]$, and the corresponding projections.

E6 If $\Delta \equiv \Gamma, v_1\colon B_1, \ldots, v_m\colon B_m, v_{m+1}\colon C$, for any morphism $[\langle v_1, \ldots, v_m, v_{m+1}, p\rangle_\Gamma]\colon [\Delta] \to [\Delta, x\colon \mathrm{Prop}]$ we add

$$[\langle v_1, \ldots, v_m, (\forall\colon v_{m+1}C)p\rangle_\Gamma]\colon [\Gamma, v_1\colon B_1, \ldots, v_m\colon B_m] \to [\Gamma, v_1\colon B_1, \ldots, v_m\colon B_m, x\colon \mathrm{Prop}]$$

$$[\Delta, x\colon \mathrm{Proof}(p)]$$

and its corresponding projection.

E7 If $\Delta \equiv \Gamma, v_1 : B_1, \ldots, v_m : B_m$, for every morphism of the form $\langle v_1, \ldots, v_m, t \rangle_\Gamma : [\Delta] \to [\Delta, f : (\Pi x : A)B]$ and every morphism $\langle v_1, \ldots, v_m, s \rangle_\Gamma : [\Delta] \to [\Delta, x : A]$, such that $[\Delta, y : B[x \leftarrow s]]$ exists in G_n, add

$$[\langle v_1, \ldots, v_m, \mathrm{App}([x : A]B, t, s) \rangle_\Gamma] : [\Delta] \to [\Delta, y : B[x \leftarrow s]]$$

After each expansion, we complete the category in the following way: for every product object, i.e., for each object $[\Delta]$ with Δ of the form $\Omega, (\Pi x : A)B$, and every section $s = \langle x, a \rangle_\Omega$ of $[\Omega, x : A, y : B]$ we add $\mathrm{curry}(s) = \langle (\lambda x : A)a \rangle_\Omega$ as a section of $[\Delta]$. Also, we add the composition of morphisms to G_{n+1} in order to complete it as a subcategory of $\mathcal{C}_{\Sigma,\Gamma}$. As in the case of GATs, when adding a morphism we are implicitly adding its endpoints, and by adding an object we are adding its projection.

Now we prove the exhaustivity theorem. In order to do it we will have to prove some previous lemmas:

First, we introduce inversion (or generation) lemmas: given a derivable sequent, they characterize the syntactic form of its principal expression. The proof method is simultaneous structural induction on derivation height, and is detailed in Appendix B

Lemma 2.14. *Given a derivable sequent of the form $\Gamma \vdash A$ type there are 4 possible cases:*

1. *A is equal to Prop;*
2. *A is equal to Proof(p) for some pre-object-expression p and $\Gamma \vdash p : Prop$ is derivable;*
3. *A is equal to $(\Pi x : B)C$ for pre-type-expressions B, C and a variable x, such that $\Gamma, x : B \vdash C$ type is a derivable sequent;*
4. *There exists a sort symbol S in Σ with an introductory rule $y_1 : B_1, \ldots, y_m : B_m \vdash S$ type and sequents*

$$\begin{aligned} &\Gamma \vdash t_1 : B_1 \\ &\Gamma \vdash t_2 : B_2[y_1 \leftarrow t_1] \\ &\qquad \vdots \\ &\Gamma \vdash t_m : B_m[y_1 \leftarrow t_1, \ldots, y_{m-1} \leftarrow t_{m-1}] \end{aligned}$$

such that A is equal to $S[y_1 \leftarrow t_1, \ldots, y_m \leftarrow t_m]$.

Lemma 2.15. *Given a derivable sequent of the form $\Gamma \vdash t : A$ there are 5 possible cases:*

1. *there exists a variable x in Var(Γ), such that t equals x;*

2. *t is equal to $(\lambda x\colon B)s$ and A is equal to $(\Pi x\colon B)C$ for some pre-type-expressions B, C, a pre-object-expression s and variable x;*

3. *t is equal to $App([x\colon B]C, s, r)$ and A is equal to $C[x \leftarrow r]$ for some pre-type-expressions B, C, pre-object-expressions s, r and variable x;*

4. *t is equal to $(\forall x\colon B)s$ and A is equal to Prop for some pre-type-expression B, pre-object-expression s and variable x;*

5. *there exist an operator symbol F in Σ with an introductory rule $y_1\colon B_1, \ldots, y_m\colon B_m \vdash F\colon B$ and sequents*

$$\begin{aligned}\Gamma &\vdash t_1\colon B_1\\ \Gamma &\vdash t_2\colon B_2[y_1 \leftarrow t_1]\\ &\vdots\\ \Gamma &\vdash t_m\colon B_m[y_1 \leftarrow t_1, \ldots, y_{m-1} \leftarrow t_{m-1}]\end{aligned}$$

such that t is equal to $F[y_1 \leftarrow t_1, \ldots, y_m \leftarrow t_m]$.

Now we expand the concept of depth to this new syntax:

Definition 2.16. By structural induction of expressions we define a function depth mapping pre-well-formed expressions and pre-contexts to ω:

1. if x is a variable, then $\text{depth}[x] = 1$;

2. $\text{depth}[\text{Prop}] = 1$;

3. if A is a pre-type-expression, then $\text{depth}[\text{Proof}(A)] = \text{depth}[A] + 1$;

4. if x is a variable, A, B are pre-type-expression and t, s are pre-type-expressions, then

$$\begin{aligned}\text{depth}[(\Pi x\colon A)B] &= \text{depth}[A] + \text{depth}[B] + 1\\ \text{depth}[(\lambda x\colon A)t] &= \text{depth}[A] + \text{depth}[t] + 1\\ \text{depth}[(\forall x\colon A)t] &= \text{depth}[A] + \text{depth}[t] + 1\\ \text{depth}[\text{App}[x\colon A]B, t, s] &= \text{depth}[A] + \text{depth}[B] + \text{depth}[t] + \text{depth}[s] + 1;\end{aligned}$$

5. if S is a sort symbol from Σ with an introductory rule $z_1\colon C_1, \ldots z_w\colon C_w \vdash S$ type, and $t_1, \ldots, t_w$ are pre-term-expressions, then

$$\text{depth}[S[z_1 \leftarrow t_1, \ldots, z_w \leftarrow t_w]] = 1 + \sum_{i=0}^{w} \text{depth}[t_i];$$

6. if F is an operator symbol from Σ with an introductory rule $z_1 : C_1, \ldots, z_w : C_w \vdash F : A$, and $t_1, \ldots, t_w$ are pre-term-expressions, then

$$\mathrm{depth}[F[z_1 \leftarrow t_1, \ldots, z_w \leftarrow t_w]] = 1 + \sum_{i=0}^{w} \mathrm{depth}[t_i]$$

We use it to prove the following lemma:

Lemma 2.17. *Every object and every section in $\mathcal{C}_{\Sigma,\Gamma}$ are constructible by expansion.*

Proof. We perform this proof by induction over the degree of the objects and sections, where

1. if Δ is a context over Γ, then

$$\mathrm{degree}[\Delta] = \mathrm{depth}[\Delta] - \mathrm{depth}[\Gamma]$$

2. if $\Delta \equiv \Gamma, y_1 : B_1, \ldots, y_m : B_m$ is a context and $[\langle y_1, \ldots, y_m, t \rangle_\Gamma]$ is a section of $[\Delta, x : A]$, then

$$\mathrm{degree}[\langle y_1, \ldots, y_m, t \rangle_\Gamma] = \mathrm{degree}[\Delta] + \mathrm{depth}[t]$$

Consider then a context $\Delta \equiv \Gamma, y_1 : B_1, \ldots, y_m : B_m$. We want to see that $[\Delta]$ is constructible by expansion. If $\mathrm{degree}[\Delta] = 0$, then it is the case $\Delta = \Gamma$, and hence it is constructible by expansion. Consider now the case $\mathrm{degree}[\Delta] > 0$. As the sequent Δ ok is derivable, by the rule Cont-Intro, we have that $\Delta' \vdash B_m$ type is derivable, where $\Delta' = \Gamma, y_1 : B_1, \ldots, y_{m-1} : B_{m-1}$. By Lemma 2.14, we have to consider four cases:

1. We have that $B_m = \mathrm{Prop}$. By induction hypothesis, $[\Delta']$ is constructible by expansion, also we have that $[\Gamma]$ and $[\Gamma, p : \mathrm{Prop}]$ are constructible by expansion. Then by applying the rule E4 on the object $[\Gamma, p : \mathrm{Prop}]$ and the projection morphism of $[\Delta']$ on $[\Gamma]$, we have that the following pullback diagram is constructible by expansion:

$$\begin{array}{ccc} [\Delta', x : \mathrm{Prop}] & \xrightarrow{\langle x \rangle_\Gamma} & [\Gamma, x : \mathrm{Prop}] \\ \downarrow & & \downarrow \\ [\Delta'] & \longrightarrow & [\Gamma] \end{array}$$

and $[\Delta', x : \mathrm{Prop}] = [\Delta]$ as they only differ by the naming of the bound variables.

2. We have that $B_m = \mathrm{Proof}(p)$ for some pre-type-expression p. By rule Prop-Type, we have that $\Delta' \vdash p : \mathrm{Prop}$. By induction hypothesis, $[\Delta', x : \mathrm{Prop}]$ and

the section $[\langle y_1, \ldots, y_{m-1}, p\rangle_\Gamma] \colon [\Delta'] \to [\Delta', x \colon \text{Prop}]$ are constructible by expansion. Then, by applying expansion rule E6, we have that $[\Delta', x \colon \text{Proof}(p)]$ is constructible by expansion, and so it is $[\Delta] = [\Delta', x \colon \text{Proof}(p)]$ as they differ only by the naming of the bound variables.

3. We have that $B_m = (\Pi x \colon A)B$ for pre-type-expressions A, B and a variable x. By induction hypothesis,

$$[\Delta'] \longleftarrow [\Delta', x \colon A] \longleftarrow [\Delta', x \colon A, y \colon B]$$

is constructible by expansion. By applying expansion rule E5 to $[\Delta', x \colon A, y \colon B]$ we have that

$$\Pi([\Delta, x \colon A, y \colon B]) = [\Delta', x \colon (\Pi x \colon A)B]$$

is constructible by expansion. Then, so is $[\Delta] = [\Delta', x \colon (\Pi x \colon A)B]$ as they only differ on the naming of bound variables.

4. We have that there exists a sort symbol S in Σ with an introductory rule $z_1 \colon C_1, \ldots, z_w \colon C_w \vdash S$ type and sequents

$$\begin{aligned}
&\Gamma \vdash t_1 \colon C_1 \\
&\Gamma \vdash t_2 \colon C_2[z_1 \leftarrow t_1] \\
&\qquad\vdots \\
&\Gamma \vdash t_w \colon C_w[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}]
\end{aligned}$$

such that B_m is equal to $S[z_1 \leftarrow t_1, \ldots, z_w \leftarrow t_w]$. By induction hypothesis, we have that the following sections are constructible by expansion:

$$\begin{aligned}
&[\langle y_1, \ldots, y_{m-1}, t_1\rangle_\Gamma] \colon [\Delta'] \to [\Delta', z_1 \colon C_1] \\
&[\langle y_1, \ldots, y_{m-1}, t_2\rangle_\Gamma] \colon [\Delta'] \to [\Delta', z_2 \colon C_2[z_1 \leftarrow t_1]] \\
&\qquad\vdots \\
&[\langle y_1, \ldots, y_{m-1}, t_w\rangle_\Gamma] \colon [\Delta'] \to [\Delta', z_w \colon C_w[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}]]
\end{aligned}$$

So by applying the expansion rule E1 to $[\Delta']$, we have that $[\Delta', x \colon S[z_1 \leftarrow t_1, \ldots, z_w \leftarrow t_w]]$ is constructible by expansion. Hence, so is $[\Delta] = [\Delta', x \colon S[z_1 \leftarrow t_1, \ldots, z_w \leftarrow t_w]]$ as they differ only in the naming of bound variables.

Now consider the section

$$[\langle y_1, \ldots, y_m, t\rangle_\Gamma] \colon [\Delta] \to [\Delta, x \colon A].$$

We want to see that it is constructible by expansion. We have that $\Delta \vdash t \colon A$ is derivable. By Lemma 2.15 we have the following cases:

1. Then, there is an i, $1 \leq i \leq m$, such that $t = y_i$. By induction hypothesis,

$[\Delta]$ is constructible by expansion. By applying the expansion rule E2 to $[\Delta]$, we have that the section $[\langle y_1, \ldots, y_m, y_i \rangle_\Gamma] \colon [\Delta] \to [\Delta, x \colon B_i]$ is constructible by expansion, and so is $[\langle y_1, \ldots, y_m, t \rangle_\Gamma] = [\langle y_1, \ldots, y_m - 1, y_i \rangle_\Gamma]$.

2. We have that $t = (\lambda x \colon B)s$ and $A = (\Pi x \colon B)C$ for some pre-type-expressions B, C, a pre-object-expression s and a variable x. By induction hypothesis, we have that $[\Delta, x \colon A, y \colon B]$ and its section $[\langle y_1, \ldots, y_m, x, s \rangle_\Gamma]$ are constructible by expansion. Then, by applying curry completion, we have that $[\langle y_1, \ldots, y_m, (\lambda x \colon B)s \rangle_\Gamma] = [\langle y_1, \ldots, y_m, t \rangle_\Gamma]$.

3. We have that $t = \mathrm{App}([x \colon B]C, s, r)$ and $A = C[x \leftarrow r]$ for some pre-type-expressions B, C, pre-object-expressions s, r and a variable x. By induction hypothesis, we have that $[\langle y_1, \ldots, y_m, s \rangle_\Gamma] \colon [\Delta] \to [\Delta, f \colon (\Pi x \colon B)C]$ and $[\langle y_1, \ldots, y_m, r \rangle_\Gamma] \colon [\Delta] \to [\Delta, x \colon B]$ are constructible by expansion. Then, by applying the expansion rule E7 we have that

   $$[\langle y_1, \ldots, y_m, t \rangle_\Gamma] = [\langle y_1, \ldots, y_m, \mathrm{App}([x \colon B]C, s, r) \rangle_\Gamma] \colon [\Delta] \to [\Delta, y \colon B[x \leftarrow r]]$$

   is constructible by expansion.

4. We have that $t = (\forall x \colon B)s$ and $A = \mathrm{Prop}$ for some pre-type-expression B, pre-object-expression s and variable x. By induction hypothesis, we have that $[\langle y_1, \ldots, y_m, x, s \rangle_\Gamma] \colon [\Delta, x \colon B] \to [\Delta, x \colon B, p \colon \mathrm{Prop}]$ is constructible by expansion. And by applying the expansion rule E6 the morphism

   $$[\langle y_1, \ldots, y_m, (\forall x \colon B) \rangle_\Gamma] \colon [\Delta] \to [\Delta, x \colon \mathrm{Prop}]$$

   is constructible by expansion.

5. Then, there exists an operator symbol F in Σ with an introductory rule $z_1 \colon C_1, \ldots, z_m \colon C_m \vdash F \colon B$ and sequents

   $$\begin{aligned} &\Gamma \vdash t_1 \colon C_1 \\ &\Gamma \vdash t_2 \colon C_2[z_1 \leftarrow t_1] \\ &\qquad \vdots \\ &\Gamma \vdash t_w \colon C_w[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}] \end{aligned}$$

   such that t is equal to $F[z_1 \leftarrow t_1, \ldots, z_w \leftarrow t_w]$. By induction hypothesis, we

have that the following sections are constructible by expansion:

$$[\langle y_1, \ldots, y_{m-1}, t_1 \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_1 \colon C_1]$$
$$[\langle y_1, \ldots, y_{m-1}, t_2 \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_2 \colon C_2[z_1 \leftarrow t_1]]$$
$$\vdots$$
$$[\langle y_1, \ldots, y_{m-1}, t_w \rangle_\Gamma] \colon [\Delta] \to [\Delta, z_w \colon C_w[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}]]$$

So by applying the expansion rule E3 to $[\Delta]$, we have that $[\langle y_1, \ldots, y_m, F[z_1 \leftarrow t_1, \ldots, z_w \leftarrow t_w] \rangle_\Gamma] = [\langle y_1, \ldots, y_m, t \rangle_\Gamma]$ is constructible by expansion.

□

Theorem 2.18. *Every object and every morphism of $\mathcal{C}_{\Sigma,\Gamma}$ are constructible by expansion.*

Proof. Follows directly from Lemma 2.17 and the proof of Theorem 1.19. □

Chapter 3

Implementation of the Expansion Algorithm

In this chapter we explain the basic ideas of how to use the expansion algorithm jointly with graph neural networks with the objective of generating terms of a specific type in a given theory. We also present a demo of the raw expansion algorithm for generating terms on simply typed combinatory logic in order to give some intuition on how the algorithm works.

3.1 Term Generation in Simply Typed Combinatory Logic

Now we present the IMPLICA package for Python [11], a graph engine built on Rust that we built for the purpose of this demonstration. The idea of this package is to allocate the finite subcategory G described in the expansion algorithm, and help to apply the different expansion iterations via its cypher-like interface directly from Python.

We start by stating what simply typed Combinatory Logic, or $\mathbf{CL}_{\to}$, is. This definition is taken from [23]. Given a countable set of type variables $P_0, P_1, P_2, \ldots$, we define the set of *simple types* $\mathcal{T}_{\to}$ as the set generated by the following clauses:

1. type variables belong to $\mathcal{T}_{\to}$;
2. if $A, B \in \mathcal{T}_{\to}$, then $(A \to B) \in \mathcal{T}_{\to}$.

Types of the form $A \to B$ are called *function types*, and are the simply typed version of the product types we introduced in CoC. Terms of $\mathbf{CL}_{\to}$ must have a type assigned and are generated by the following clauses:

1. for each $A \in \mathcal{T}_{\to}$ there is a countably infinite supply of variables of type A;

2. if t, s are terms of types $A \to B$ and A respectively, then $\mathrm{App}(t,s)$ is a term of type B;

3. for all $A, B, C \in \mathcal{T}_{\to}$ there are constant terms

$$\mathbf{k}^{A,B} \colon A \to B \to A$$

$$\mathbf{s}^{A,B,C} \colon (A \to B \to C) \to (A \to B) \to A \to C.$$

We use the following notation: To assert that t is a term of type A, we write $t \colon A$; instead of writing $\mathrm{App}(t,s)$ we can simply write (ts). In the case of iterated implications we sometimes use the short notation

$$A_1 \to A_2 \to \dots A_{n-1} \to A_n \quad \text{for} \quad A_1 \to (A_2 \to \dots (A_{n-1} \to A_n) \dots).$$

Similarly, for iterated application we write

$$t_1 t_2 \dots t_n \quad \text{for} \quad (\dots((t_1 t_2) t_3 \dots) t_n).$$

We can write this as a GAT signature as follows:

$$\begin{aligned}
&\vdash \mathrm{Prop}\ \mathrm{type} \\
p \colon \mathrm{Prop} &\vdash \mathrm{Proof}(p)\ \mathrm{type} \\
A, B \colon \mathrm{Prop} &\vdash (A \to B) \colon \mathrm{Prop} \\
A, B \colon \mathrm{Prop}, t \colon \mathrm{Proof}(A \to B), s \colon \mathrm{Proof}(A) &\vdash \mathrm{App}(t,s) \colon \mathrm{Proof}(B) \\
A, B \colon \mathrm{Prop} &\vdash \mathbf{k}^{A,B} \colon \mathrm{Proof}(A \to B \to A) \\
A, B, C \colon \mathrm{Prop} &\vdash \mathbf{s}^{A,B,C} \colon \mathrm{Proof}((A \to B \to C) \to (A \to B) \to A \to C)
\end{aligned}$$

This allows us to build upon the construction of $C(U)$ we made for the general case, but this will be rather difficult to follow. Hence, we will look for simplification both of the category and the expansion rules. Consider Γ a simple type expression of $\mathbf{CL}_{\to}$, let $A_1, \dots, A_n$ be the type variables appearing on Γ, and let Δ be the context $A_1 \colon \mathrm{Prop}, \dots, A_n \colon \mathrm{Prop}$. We center our search on the relativization of $C(U)$ by Δ, $\mathcal{C}_\Delta$. Although the category $\mathcal{C}_\Delta$ carries a lot of information, i.e., its tree structure, dependent types and pullbacks, in this simple case it becomes somehow redundant. We then consider the full category composed by the first level of objects and the terminal object, this subcategory contains all possible types in $A_1, \dots, A_n$ and the relations between them. Moreover, the object $[\Delta, x \colon \mathrm{Prop}]$ lies in the level 1, but it carries no important information in our case, as its just used for object formation by the general expansion algorithm, but that will be done differently in this special case, so we will remove it. So we can consider $\mathcal{C}$ to be the full subcategory of $C(U)$ containing $[\Delta]$ and $[\Delta, x \colon \mathrm{Proof}(p)]$ for each term p of type Prop under context Δ. The objects of this category are in 1-1-correspondence

with the elements of $\mathcal{T}_{\to} \cup \{1\}$, hence we note

$$1 := [\Delta] \quad \text{and} \quad p := [\Delta, x\colon \text{Proof}(p)].$$

In the case of morphisms, we will note them by the term appearing in the $n+1$ component, i.e., each morphism $\langle A_1, \ldots, A_n, s\rangle$ in $\mathcal{C}$ will be noted as just s. With this notation in mind we define the following rules of expansion: Let p be an element of $\mathcal{T}_{\to}$,

1. if p is of the form $B \to A$, we add
$$A \xrightarrow{\mathbf{k}^{A,B}} p;$$
2. for each q in G_n we add
$$p \xrightarrow{\mathbf{k}^{p,q}} q \to p;$$
3. if p is of the form $(B \to A) \to A \to C$, we add
$$(A \to B \to C) \xrightarrow{\mathbf{s}^{A,B,C}} p;$$
4. if p is of the form $(A \to B \to C)$, we add
$$p \xrightarrow{\mathbf{s}^{A,B,C}} ((A \to B) \to A \to C);$$

Also, when creating an object p, if p is of the form $A \to B \to A$, add the morphism $\mathbf{k}^{A,B}\colon 1 \to p$, and if p is of the form $(A \to B \to C) \to (A \to B) \to A \to C$, add the morphism $\mathbf{s}^{A,B,C}\colon 1 \to p$. Then, complete G_{n+1} with the composition of morphisms in order for it to be a subcategory of $\mathcal{C}$. The exhaustivity of this expansion algorithm is easy to check, as it follows directly from the definitions.

Now we show how to code the expansion algorithm using Python and the IMPLICA package. One peculiarity we developed into IMPLICA that is not part of our current definition is the fact that all terms of a type are considered equal, as we just care about having a witness, not the nature of it. In order to add this condition to the GAT signature, it is enough to add the following equation:

$$p\colon \text{Prop}, t_1, t_2\colon \text{Proof}(p) \vdash t_1 = t_2$$

As it is not really part of combinatory logic, we left it out of its formal definition, but for our task it is more efficient to consider it this way.

First we initialize an IMPLICA graph and we add the proposition $A \to A$ using the query API.

```
import implica

# Initialize the graph with Combinatory Logic constants
```

```
graph = implica.Graph(
    constants = [
        implica.Constant(("K", "(A:*)->(B:*)->A")),
        implica.Constant("S", "((A:*)->(B:*)->(C:*))->(A->B)->A->C")
    ]
)

# Seed the graph with our target objective (A -> A)
graph.query().create("(:A->A)").execute()
```

Now, to perform one expansion iteration we execute the following code:

```
# Mark all current nodes as 'existed' to differentiate them from new generations
graph.query().match("(N:*)").set("N", { "existed" : True }).execute()

# Rule 1 & 2: K-Combinator Expansions
(
    self.graph.query()
    # Find existing nodes matching the signature of a K-target
    .match("(N:(B:*) -> (A:*) { existed: true })")
    # Create the precursor node and the connecting morphism
    .create("(: A { existed: false })-[::@K(A, B)]->(N)")
    .execute()
)
(
    self.graph.query()
    .match("(N:(A:*) { existed: true })")
    # CRITICAL: Limit creation number to prevent memory overflow
    .limit(10)
    .match("(M:(B:*) { existed: true })")
    .create("(N)-[::@K(A, B)]->(:B -> A { existed: false })")
    .execute()
)
# Rule 3 & 4: S-Combinator Expansions
(
    self.graph.query()
    .match("(N:((A:*) -> (B:*)) -> A -> (C:*) { existed: true })")
    .create("(:A->B->C { existed: false })-[::@S(A, B, C)]->(N)")
    .execute()
)
(
```

```
        self.graph.query()
        .match("(N:(A:*)->(B:*)->(C:*) { existed: true })")
        .create("(N)-[::@S(A, B, C)]->(:(A -> B) -> A -> C { existed: false })")
        .execute()
    )
```

In the first line, we set an attribute "existed" to true in order to distinguish between new nodes and nodes already present at the beginning. It is important to state the *.limit* modifier on the second query, because this expansion rule creates, or tries to create a new node for each pair of nodes in the graph, hence it makes complexity explode. To solve this we use the modifier *.limit* to cap the number of creations per original node to 10. Feel free to remove this line. This explosive growth of the complexity of the algorithm is what we will try to solve later by adding graph neural networks.

Let us walk through, iteration by iteration, on the generation of a term of type $A \to A$ by the expansion algorithm to get some intuition. For simplicity, we have omitted the 1 element on the graph displays, and also, as we just want witnesses of the existence of a term, we just have at most one morphism between two objects, and the sections (morphisms from 1) will be displayed simply as red dots instead of blue. Appendix C contains images of the graphs we will be mentioning now. The starting point G_0 is rather uninteresting, as it just contains the object $A \to A$ with no section. In the Figure C.1a, we can see that G_1 contains three unpopulated objects A, $A \to A$ and $(A \to A) \to A \to A$ from left to right. Then, by applying another expansion, we obtain G_2, Figure C.1b, where we start to see some objects getting populated, for example $(A \to A) \to A \to A$ with term **sk**. By the next expansion, G_3 grows significantly, see Figure C.1c, but $A \to A$ remains unpopulated. Finally, in G_4 $A \to A$ gets inhabited by the term **skk**, but as can be seen in Figure C.1d, the exponential growth of the graph continues.

Iteration (G_n)	**Nodes (Capped at 10/match)**	**Nodes (Uncapped)**
G_0	1	1
G_1	3	3
G_2	11	11
G_3	116	123
G_4	1352	15131

Table 3.1: Node count explosion during the expansion algorithm for the proof of $A \to A$ in $CL_{\to}$.

Now that we have a clearer understanding of how the expansion algorithm

works, we introduce the optimizations for the general case.

3.2 Graph Neural Networks

Graph Neural Networks (GNNs) process complex data structures by leveraging relationships between individual data points (nodes) and their connections (edges) through a mechanism known as "message passing". During this process, each node iteratively updates its own representation by gathering and aggregating information from its immediate neighbors, enabling the model to learn both the local features of the data and the broader structural geometry of the network. Because of this unique architecture, GNNs are highly effective for node classification (assigning specific labels to individual nodes), graph classification (categorizing the entire network structure as a single entity), and graph embedding (translating complex graph structures into lower-dimensional vectors for easier processing in downstream machine learning tasks). [26] [27]

The idea behind the expansion algorithm comes from trying to use GNNs to take advantage of the graph structures that emerge from the categorical semantics we presented in the previous chapters. So we built it to work together with a GNN performing node classification. First consider the GNN as just a simple labeler, that given the finite subcategory G_n produced by the previous expansion step, the GNN labels each node in the following 3 categories: KEEP, EXPAND and DELETE. Then we define a partial expansion phase where we compute G_{n+1} from G_n without those nodes labeled as DELETE, and by only applying expansion on those nodes marked as EXPAND. This change will make us lose the exhaustivity of the process, but we will be gaining a lot of efficiency if the GNN manages to prune all the useless nodes and expand only those nodes we really need for our generation. In the ideal case, i.e. the GNN always labels nodes correctly, we will be able to still generate terms of any type but we would make it with far less complexity.

Although the implementation of the following algorithm is left for further work, we present the basic ideas for building our labeling model. The first thing we must note is that using supervised learning techniques (that need to have input-output pairs) need for us to be able to generate a large quantity of training examples by the execution of the raw expansion algorithm, hence we have to discard this option as we would just be able to train our model on very simple examples, due to the high complexity of the algorithm. Similar constraints are stated in [25]. To solve this, we would need to train using unsupervised learning. The technique we are going to use is called Teacher-Student or Conjecturer-Prover [19] [22] [9]. It consists of having a two-headed model, i.e., a model with a GNN based

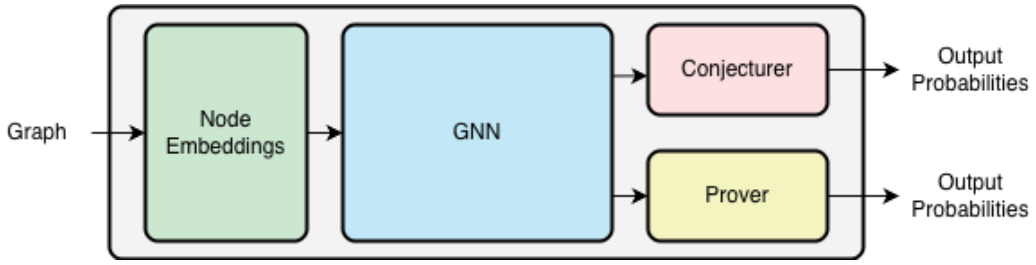


Figure 3.1: Base architecture diagram

body together with two different projection heads. The first head or conjecturer will propose a challenge, or in our case a type we need to find a term of, and the second head or prover will act as the GNN we talked before and try to find the term with the partial expansion algorithm. Then, if a solution is found, we compute the optimal labels at each step to compute the loss of the prover head. Then we compute the loss of the conjecturer by measuring how far it was the prover's loss from some objective loss. This way we make the conjecturer output problems with the right level of complexity.

We study the components separately:

Node Embedding: The objective of this component is to assign numerical embeddings, i.e., vectors, to the nodes of the input graph. This embeddings represent the raw concept of the node, without taking the graph structure into consideration. Hence it mainly reflects the syntactic meaning of the node. They can be learned or computed from the syntax. If we fix a signature, we could learn a basic embeddings for all sort and operator symbols of the signature, assign variables a non-learnable embedding computed just from the name, and learn some embedding merging operations to generate embeddings of more complex expressions or contexts. This merging operations can be performed by small encoder only transformer models for example. In the case were we have no fixed signature, i.e., we want to create a foundational model to be used with any given signature we could just learn the aggregation operations, but this is left for further research.

GNN Body: The exact architecture of the message passing GNN has to be determined, but it's propose is to enrich the basic embeddings into structure aware embeddings. It is important that the architecture selected expects directed edges, as the direction of morphisms is key in this setup.

Conjecturer: Just a linear projection layer from the original embedding dimension to a 3-dimensional vector. This means that the conjecturer will be classifying nodes into 3 categories: Objective, Include and Exclude, with the constraint that only one node can be chosen to be the objective node, so first we take the node with the highest Objective score, mark it as an Objective, then we perform

soft-max with the other two components of the other nodes to assign the labels INCLUDE and EXCLUDE. Then the challenge the Prover will try to solve is to generate a section of the OBJECTIVE node starting from the graph composed by the OBJECTIVE node and all the nodes labeled as INCLUDE. Morphisms that exist due to composition or currying of morphisms that include an excluded node won't be included.

Prover: this component is just a linear layer from the original embedding dimension to a 3-dimensional vector, same as the conjecturer, and then apply a soft-max. It classifies the nodes in 3 categories: KEEP, EXPAND and DELETE. In the case when the objective isn't distinguished in an earlier phase, we could consider passing both the embeddings of the node together with the embedding of the objective node to this component.

Now we have to consider training the model. The main problem is cold start. At the beginning we face an empty graph to feed to the conjecturer and a conjecturer yielding nonsense. This means that we have to somehow pre-train the conjecturer before stating the unsupervised training loop. We will propose a way to go around this problem, but it must undergo testing before assuring it works. We start by feeding the model a type we know it will find a type in relatively few steps of expansion, like $A \to A$ in our previous example. We can use this expansion to construct a base graph. This graph contains objects whose sections are fairly easy to construct, as we did it in a few iterations, then we select some of them and use them first as labeled examples for our conjecturer. Then we start the unsupervised training loop. At first, our prover also yields nonsense, so if we let it decide the path of our partial expansion it will fail catastrophically, leaving us with no examples to train our model with. To solve this, we consider a function that determines the score (value) needed by a label in order for us to take action. Name this function $f\colon \omega \to (0, +\infty)$. As an example, we take the function $f = 0.8$, then if we get the output $(0.3, 0.3, 0.4)$ from the prover, that node would still be label EXPAND, although it should be labeled as DELETE. In the other hand, if the prover output $(0.05, 0.1, 0.85)$ for a node, it would be labeled as DELETE. A constant function provides little-to-none benefit, because it will cap the complexity of objects our model can handle, as by not allowing drastic pruning the model won't be able to dig to deep into the graph. An example of such a function could be $f(x) = (a - b)\exp^{-cx} + b$, with $a, b, c > 0$. This confidence baseline $f(x)$ will decay over time, when x grows, and it will give more autonomy to the Prover as it learns to accurately slice through the combinatorial explosion.

Conclusion

This thesis develops a categorical approach to autonomous theorem proving, grounded in the translation of type inhabitation problems into geometric graph exploration problems over term models.

Our investigation began with the study of Lafforgue's framework connecting Topos Theory [15] with artificial intelligence. While Toposes offered a compelling theoretical foundation, they proved too abstract for direct application. This led us to Cartmell's Generalized Algebraic Theories and contextual categories [4] [5], whose term models sit close enough to the syntax of a theory to be computationally manageable, while still carrying the geometric structure needed for a GNN-based approach. We proved the initiality conjecture for GATs in Section 1.6, establishing C(U) as the canonical categorical representation of the semantics of a theory, and designed the Expansion Algorithm to construct sections within it, equivalent to generating terms of a given type.

To handle a more expressive type theory, we turned to Streicher's work on the Calculus of Constructions and Doctrines of Constructions [21], which provided both a proof of the initiality conjecture for CoC and a framework for extending type theories beyond GATs. In Section 2.5, we extended this framework to CoC enriched with a GAT signature, constructed the corresponding term model, and proved its initiality following ideas from [2]. We then extended the Expansion Algorithm to this richer setting, to do this we extend the concept of relativization to Doctrines of Constructions. In the course of this work, we also identified a connection between the concept of collections of types in Doctrines of Constructions and the topos-theoretic notion of subobject classifier [18], which we leave as a direction for further investigation.

Several directions remain open. On the theoretical side, it remains to formally extend the term model construction and prove the initiality conjecture for Doctrines of Constructions with Î£-types, identity types, and a universe hierarchy, as described in [21], and ultimately for the full Calculus of Inductive Constructions [7] [20] and the Lean type theory [8]. The initiality conjecture for full Martin-LÃ¶f type theory has been formalized in Agda by Brunerie and Lumsdaine [3], and a

similar formalization effort for our extended setting would be a natural next step.

On the practical side, the full implementation of the GNN-based Conjecturer-Prover model for guiding the Expansion Algorithm remains the primary target for future work. We believe this architecture, by combining the geometric structure of categorical term models with the pruning power of graph neural networks, offers a principled and promising alternative to purely syntactic approaches to autonomous theorem proving.

Appendix A

Derivation Rules

We denote "from $S_1, \ldots, S_n$ derive S", where $S_1, \ldots, S_n, S$ are rules or sequents, as follows:

$$\frac{S_1 \quad \ldots \quad S_n}{S}$$

Also, we write

$$\frac{S_2}{\overline{S_1}} \quad \text{abbreviates} \quad \frac{S_1}{S_2} \text{ and } \frac{S_2}{S_1}$$

A.1 Derivation Rules of Generalized Algebraic Theories

In this section we present the derivation rules of Generalized Algebraic Theories as presented in [5], but with a more compact notation.

$$\text{LI1}\ \frac{\Gamma \vdash A \text{ type}}{\Gamma \vdash A = A} \qquad \text{LI2}\ \frac{\Gamma \vdash t : A}{\Gamma \vdash t = t : A} \qquad \text{LI3}\ \frac{\Gamma \vdash A_1 = A_2}{\Gamma \vdash A_2 = A_1}$$

$$\text{LI4}\ \frac{\Gamma \vdash t_1 = t_2 : A}{\Gamma \vdash t_2 = t_1 : A} \qquad \text{LI5}\ \frac{\Gamma \vdash A_1 = A_2 \qquad \Gamma \vdash A_2 = A_3}{A_1 = A_3}$$

$$\text{LI6}\ \frac{\Gamma \vdash t_1 = t_2 : A \qquad \Gamma \vdash t_2 = t_3 : A}{\Gamma \vdash t_1 = t_3 : A} \qquad \text{LI7}\ \frac{\Gamma \vdash t_1 = t_2 : A_1 \qquad \Gamma \vdash A_1 = A_2}{t_1 = t_2 : A_2}$$

$$\text{T1}\ \frac{\Gamma \vdash A_1 = A_2 \quad \Gamma \vdash t : A_1}{\Gamma \vdash t : A_2}$$

CF1 For $n \geq 0, 1 \leq i \leq n+1$:

$$\frac{x_1 \colon A_1, \ldots, x_n \colon A_n \vdash A_{n+1} \text{ type}}{x_1 \colon A_1, \ldots, x_{n+1} \colon A_{n+1} \vdash x_i \colon A_i}$$

providing that x_{n+1} is a variable distinct from all $x_1, \ldots, x_n$.

CF2(A) For every sort symbol S with well-formed introductory rule

$$x_1 \colon A_1, \ldots, x_n \colon A_n \vdash S(x_1, \ldots, x_n) \text{ type},$$

for every context Γ,

$$\frac{\Gamma \vdash t_1 \colon A_1 \quad \Gamma \vdash t_2 \colon A_2[x_1 \leftarrow t_1] \quad \ldots \quad \Gamma \vdash t_n \colon A_n[x_1 \leftarrow t_1, \ldots, x_{n-1} \leftarrow t_{n-1}]}{\Gamma \vdash S(t_1, \ldots, t_n) \text{ type}}$$

CF2(B) For every operator symbol F with well-formed introductory rule

$$x_1 \colon A_1, \ldots, x_n \colon A_n \vdash F(x_1, \ldots, x_n) \colon A,$$

for every context P,

$$\frac{\Gamma \vdash t_1 \colon A_1 \quad \Gamma \vdash t_2 \colon A_2[x_1 \leftarrow t_1] \quad \ldots \quad \Gamma \vdash t_n \colon A_n[x_1 \leftarrow t_1, \ldots, x_{n-1} \leftarrow t_{n-1}]}{\Gamma \vdash F(t_1, \ldots, t_n) \colon A[x_1 \leftarrow t_1, \ldots, x_n \leftarrow t_n]}$$

SI1 If Γ is a context, then

$$\frac{\begin{array}{c} x_1 \colon A_1, \ldots x_n \colon A_n \vdash A = A' \\ \Gamma \vdash s_1 = s_1' \colon A_1 \quad \Gamma \vdash s_2 = s_2' \colon A_2[x_1 \leftarrow s_1] \\ \ldots \quad \Gamma \vdash s_m = s_m' \colon A_n[x_1 \leftarrow s_1, \ldots, x_{n-1} \leftarrow s_{n-1}] \end{array}}{\Gamma \vdash A[x_1 \leftarrow s_1, \ldots, x_n \leftarrow s_n] = A'[x_1 \leftarrow s_1', \ldots, x_n \leftarrow s_n']}$$

SI2 If Γ is a context, then

$$\frac{\begin{array}{c} x_1 \colon A_1, \ldots, x_n \colon A_n \vdash s = s' \colon A \\ \Gamma \vdash s_1 = s_1' \colon A_1 \quad \Gamma \vdash s_2 = s_2' \colon A_2[x_1 \leftarrow s_1] \\ \ldots \quad \Gamma \vdash s_n = s_n' \colon A_n[x_1 \leftarrow s_1, \ldots, x_{n-1} \leftarrow s_{n-1}] \end{array}}{s[x_1 \leftarrow s_1, \ldots, x_n \leftarrow s_n] = s'[x_1 \leftarrow s_1', \ldots, x_n \leftarrow s_n']}$$

A1 If $x_1 \colon A_1, \ldots, x_n \colon A_n \vdash A = A'$ is an axiom, then

$$\frac{x_1 \colon A_1, \ldots, x_n \colon A_n \vdash A \text{ type} \qquad x_1 \colon A_1, \ldots, x_n \colon A_n \vdash A' \text{ type}}{x_1 \colon A_1, \ldots, x_n \colon A_n \vdash A = A'}$$

A2 If $x_1 : A_1, \ldots, x_n : A_n \vdash t = t' : A$ is an axiom, then from

$$\frac{x_1 : A_1, \ldots, x_n : A_n \vdash t : A \qquad x_1 : A_1, \ldots, x_n : A_n \vdash t' : A}{x_1 : A_1, \ldots, x_n : A_n \vdash t = t' : A}$$

A.2 Derivation Rules of Calculus of Constructions

In this section we present all the derivation rules for CoC as presented in [21].

Context Formation Rules:

$$\textsc{Empty} \quad \frac{}{\text{ok}} \qquad\qquad \textsc{Cont-Intro} \quad \frac{\Gamma, x : A \text{ ok}}{\Gamma \vdash A \text{ type}}, \text{if} x \notin \text{Var}(\Gamma)$$

$$\textsc{Cont-Thin} \quad \frac{\Gamma, \Delta \text{ ok} \qquad \Gamma \vdash A \text{ type}}{\Gamma, x : A, \Delta \text{ ok}}, \text{if} x \notin \text{Var}(\Gamma) \cup \text{Var}(\Delta)$$

$$\textsc{Cont-Sub} \quad \frac{\Gamma, x : A, \Delta \text{ ok} \qquad \Gamma \vdash t : A}{\Gamma, \Delta[x \leftarrow t] \text{ ok}}$$

Context Equality Rules:

$$\textsc{Crefl} \quad \frac{\Gamma = \Gamma}{\Gamma \text{ ok}} \qquad \textsc{Csym} \quad \frac{\Gamma = \Delta}{\Delta = \Gamma} \qquad \textsc{Ctrans} \quad \frac{\Gamma = \Delta \qquad \Delta = \Theta}{\Gamma = \Theta}$$

$$\textsc{Cequ} \quad \frac{\Gamma, x : A, \Delta \text{ ok} \qquad \Gamma \vdash A = B}{\Gamma, x : A, \Delta = \Gamma, x : B, \Delta} \qquad \textsc{Creflect1} \quad \frac{\Gamma, x : A = \Delta, x : B}{\Gamma = \Delta}$$

$$\textsc{Creflect2} \quad \frac{\Gamma, x : A = \Delta, x : B}{\Gamma \vdash A = B}$$

Structural Rules for Judgement Formation

$$\textsc{Ass}\ \frac{\Gamma, x\colon A, \Delta \text{ ok}}{\Gamma, x\colon A, \Delta \vdash x\colon A} \qquad \textsc{Reflection1}\ \frac{\Gamma \vdash \mathcal{J}}{\Gamma \text{ ok}}$$

$$\textsc{Reflection2}\ \frac{\Gamma \vdash t : A}{\Gamma \vdash A \text{ type}}$$

$$\textsc{Thin}\ \frac{\Gamma, \Delta \vdash \mathcal{J} \qquad \Gamma \vdash A \text{ type}}{\Gamma, x\colon A, \Delta \vdash \mathcal{J}}, \text{ if } x \notin \mathrm{Var}(\Gamma) \cup \mathrm{Var}(\Delta)$$

$$\textsc{Sub}\ \frac{\Gamma, x\colon A, \Delta \vdash \mathcal{J} \qquad \Gamma \vdash t : A}{\Gamma, \Delta[x \leftarrow t] \vdash \mathcal{J}[x \leftarrow t]}$$

Equality Rules:

$$\textsc{Equ}\ \frac{\Gamma = \Delta \qquad \Gamma \vdash \mathcal{J}}{\Delta \vdash \mathcal{J}} \qquad \textsc{Refl-Typ}\ \frac{\Gamma \vdash A \text{ type}}{\Gamma \vdash A = A}$$

$$\textsc{Refl-Obj}\ \frac{\Gamma \vdash t : A}{\Gamma \vdash t = t : A} \qquad \textsc{Symm-Typ}\ \frac{\Gamma \vdash A = B}{\Gamma \vdash B = A}$$

$$\textsc{Symm-Obj}\ \frac{\Gamma \vdash t = s\colon A}{\Gamma \vdash s = t\colon A} \qquad \textsc{Trans-Typ}\ \frac{\Gamma \vdash A = B \qquad \Gamma \vdash B = C}{\Gamma \vdash A = C}$$

$$\textsc{Trans-Obj}\ \frac{\Gamma \vdash t_1 = t_2\colon A \qquad \Gamma \vdash t_2 = t_3\colon A}{\Gamma \vdash t_1 = t_3\colon A}$$

$$\textsc{Repl1}\ \frac{\Gamma \vdash t = s\colon A \qquad \Gamma, x\colon A, \Delta \text{ ok}}{\Gamma, \Delta[x \leftarrow t] = \Gamma, \Delta[x \leftarrow t]}$$

$$\textsc{Repl2}\ \frac{\Gamma \vdash t = s : A \qquad \Gamma, x\colon A, \Delta \vdash B \text{ type}}{\Gamma, \Delta[x \leftarrow t] \vdash B[x \leftarrow t] = B[x \leftarrow s]}$$

$$\textsc{Repl3}\ \frac{\Gamma \vdash t_1 = t_2 : A \qquad \Gamma, x\colon A, \Delta \vdash s : B}{\Gamma, \Delta[x \leftarrow t_1] \vdash s[x \leftarrow t_1] = s[x \leftarrow t_2] : B[x \leftarrow t_1]}$$

$$\textsc{Conv1}\ \frac{\Gamma \vdash A = B \qquad \Gamma \vdash t : A}{\Gamma \vdash t : B} \qquad \textsc{Conv2}\ \frac{\Gamma \vdash A = B \qquad \Gamma \vdash t = s : A}{\Gamma \vdash t = s : B}$$

Rules for products of types:

$$\Pi\text{-Form}\quad \frac{\Gamma, x\colon A \vdash B \text{ type}}{\Gamma \vdash (\Pi x\colon A)B \text{ type}}$$

$$\Pi\text{-Equ}\quad \frac{\Gamma \vdash A_1 = A_2 \qquad \Gamma, x\colon A_1 \vdash B_1 = B_2}{\Gamma \vdash (\Pi x\colon A_1)B_1 = (\Pi x\colon A_2)B_2}$$

$$\Pi\text{-Intro}\quad \frac{\Gamma, x\colon A \vdash t\colon B}{\Gamma \vdash (\lambda x\colon A)t\colon (\Pi x\colon A)B}$$

$$\xi\text{-Rule}\quad \frac{\Gamma, x\colon A_1 \vdash t_1 = t_2\colon B \qquad \Gamma \vdash A_1 = A_2}{\Gamma \vdash (\lambda x\colon A_1)t_1 = (\lambda x\colon A_2)t_2\colon (\Pi x\colon A_1)B}$$

$$\Pi\text{-Elim}\quad \frac{\Gamma \vdash t\colon (\Pi x\colon A)B \qquad \Gamma \vdash s\colon A \qquad \Gamma, x\colon A \vdash B \text{ type}}{\Gamma \vdash \mathrm{App}([x\colon A]B, t, s)\colon B[x \leftarrow s]}$$

$$\Pi\text{-Elim-Equ}\quad \frac{\begin{array}{c}\Gamma \vdash A_1 = A_2 \\ \Gamma \vdash t_1 = t_2\colon (\Pi x\colon A_1)B_1 \qquad \Gamma, x\colon A_1 \vdash B_1 = B_2 \qquad \Gamma \vdash s_1 = s_2\colon A_1\end{array}}{\Gamma \vdash \mathrm{App}([x\colon A_1]B_1, t_1, s_1) = \mathrm{App}([x\colon A_2]B_2, t_2, s_2)\colon B_1[x \leftarrow s_1]}$$

$$\beta\text{-Rule}\quad \frac{\Gamma, x\colon A \vdash B \text{ type} \qquad \Gamma, x\colon A \vdash t\colon B \qquad \Gamma \vdash s\colon A}{\Gamma \vdash \mathrm{App}([x\colon A]B, (\lambda x : A)t, s) = t[x \leftarrow s]\colon B[x \leftarrow s]}$$

$$\eta\text{-rule}\quad \frac{\Gamma, x\colon A \vdash B \text{ type} \qquad \Gamma \vdash t\colon (\Pi x\colon A)B}{\Gamma \vdash (\lambda x\colon A)\mathrm{App}([x\colon A]B, t, x) = t\colon (\Pi x\colon A)B}$$

Rules for Propositions

$$\text{Prop}\quad \frac{\Gamma \text{ ok}}{\Gamma \vdash \mathrm{Prop} \text{ type}} \qquad \text{Prop-Type}\quad \frac{\Gamma \vdash \mathrm{Proof}(p) \text{ type}}{\Gamma \vdash p\colon \mathrm{Prop}}$$

$$\text{Prop-Equ}\quad \frac{\Gamma \vdash \mathrm{Proof}(p_1) = \mathrm{Proof}(p_2)}{\Gamma \vdash p_1 = p_2\colon \mathrm{Prop}} \qquad \forall\text{-Intro}\quad \frac{\Gamma, x\colon A \vdash p\colon \mathrm{Prop}}{\Gamma \vdash (\forall x\colon A)p\colon \mathrm{Prop}}$$

$$\forall\text{-Equ}\quad \frac{\Gamma, x\colon A_1 \vdash p_1 = p_2\colon \mathrm{Prop} \qquad \Gamma \vdash A_1 = A_2}{\Gamma \vdash (\forall x\colon A_1)p_1 = (\forall x\colon A_2)p_2\colon \mathrm{Prop}}$$

$$\forall\text{-Elim}\quad \frac{\Gamma, x\colon A \vdash p \in \mathrm{Prop}}{\Gamma \vdash \mathrm{Proof}((\forall x\colon A)p) = (\Pi x\colon A)\mathrm{Proof}(p)}$$

Appendix B

Proof of Inversion Lemmas

Now we proof the inversion lemmas by simultaneous structural induction on derivation height.

Lemma B.1. *Given a derivable sequent of the form $\Gamma \vdash A$ type there are 4 possible cases:*

1. *A is equal to Prop;*
2. *A is equal to Proof(p) for some pre-object-expression p and $\Gamma \vdash p : \mathrm{Prop}$ is derivable;*
3. *A is equal to $(\Pi x : B)C$ for pre-type-expressions B, C and a variable x, such that $\Gamma, x : B \vdash C$ type is a derivable sequent;*
4. *There exists a sort symbol S in Σ with an introductory rule $y_1 : B_1, \ldots, y_m : B_m \vdash S$ type and sequents*

$$\begin{aligned} &\Gamma \vdash t_1 : B_1 \\ &\Gamma \vdash t_2 : B_2[y_1 \leftarrow t_1] \\ &\qquad \vdots \\ &\Gamma \vdash t_m : B_m[y_1 \leftarrow t_1, \ldots, y_{m-1} \leftarrow t_{m-1}] \end{aligned}$$

such that A is equal to $S[y_1 \leftarrow t_1, \ldots, y_m \leftarrow t_m]$.

Proof. The four cases are mutually exclusive by the unique outermost constructor of pre-type-expressions (the BNF grammar for E is unambiguous). We proceed by induction on $|\pi|$, with case analysis on the last rule applied.

Group I: Direct Type Formation Rules

These immediately yield the required form and witnesses.

- PROP: $\dfrac{\Gamma \text{ ok}}{\Gamma \vdash \text{Prop type}}$
  $A \equiv \text{Prop}$. This is Case 1.

- Prop-Type: $\dfrac{\Gamma \vdash p\colon \mathrm{Prop}}{\Gamma \vdash \mathrm{Proof}(p)\ \mathrm{type}}$
  $A \equiv \mathrm{Proof}(p)$. The premise $\Gamma \vdash p\colon \mathrm{Prop}$ is a strict sub-derivation. This is Case 2.
- Π-Form: $\dfrac{\Gamma, x\colon B \vdash C\ \mathrm{type}}{\Gamma \vdash (\Pi x\colon B)C\ \mathrm{type}}$
  $A \equiv (\Pi x\colon B)C$. The premise $\Gamma, x\colon B \vdash C\ \mathrm{type}$ is a strict sub-derivation. This is Case 3.
- Introductory Rule: For a sort symbol $S \in \Sigma$ with rule $z_1\colon C_1, \ldots, z_w\colon C_w \vdash S$ type,
  $$\frac{\Gamma \vdash t_1\colon C_1 \quad \cdots \quad \Gamma \vdash t_w\colon C_w[z_1 \leftarrow t_1, \ldots, z_{w-1} \leftarrow t_{w-1}]}{\Gamma \vdash S[z_1 \leftarrow t_1, \ldots, z_w \leftarrow t_w]\ \mathrm{type}}$$
  $A \equiv S[z_1 \leftarrow t_1, \ldots, z_w \leftarrow t_w]$ with all typing premises as strict sub-derivations. This is Case 4.

Group II: Context-Structural Rules

These change the context but not the outermost constructor of A; the case is transferred by applying the same rule to each sub-derivation.

- Equ: $\dfrac{\Delta = \Gamma \quad \Delta \vdash A\ \mathrm{type}}{\Gamma \vdash A\ \mathrm{type}}$
  Apply the IH to $\Delta \vdash A$ type (smaller $|\pi|$). One of the four cases holds in context Δ. Transfer each to Γ:
  - Case 1 ($A \equiv \mathrm{Prop}$): Trivially holds for Γ.
  - Case 2 ($\Delta \vdash p\colon \mathrm{Prop}$): Apply Equ with $\Gamma = \Delta$ (from $\Delta = \Gamma$ by Csym) to get $\Gamma \vdash p\colon \mathrm{Prop}$.
  - Case 3 ($\Delta, x\colon B \vdash C$ type): From $\Delta = \Gamma$ derive $\Delta, x\colon B = \Gamma, x\colon B$ by Cequ. Apply Equ to get $\Gamma, x\colon B \vdash C$ type.
  - Case 4 ($\Delta \vdash t_i\colon \ldots$ for each i): Apply Equ to each typing.
- Thin: $\dfrac{\Gamma, \Delta \vdash A\ \mathrm{type} \quad \Gamma \vdash B\ \mathrm{type}}{\Gamma, y\colon B, \Delta \vdash A\ \mathrm{type}}$
  Apply the IH to $\Gamma, \Delta \vdash A$ type. Transfer sub-derivations to $(\Gamma, y\colon B, \Delta)$ using Thin (using Cont-Thin for the extended context in Case 3). All four cases transfer.
- Sub: $\dfrac{\Gamma, y\colon B, \Delta \vdash A\ \mathrm{type} \quad \Gamma \vdash s\colon B}{\Gamma, \Delta[y \leftarrow s] \vdash A[y \leftarrow s]\ \mathrm{type}}$
  Apply the IH to $\Gamma, y\colon B, \Delta \vdash A$ type. Substitution preserves the outermost constructor.
  - Case 1: $A[y \leftarrow s] \equiv \mathrm{Prop}$.

- Case 2: $A[y \leftarrow s] \equiv \text{Proof}(p[y \leftarrow s])$. Apply Sub to $\Gamma, y\colon B, \Delta \vdash p\colon \text{Prop}$ to obtain $\Gamma, \Delta[y \leftarrow s] \vdash p[y \leftarrow s]\colon \text{Prop}$.
- Case 3: $A[y \leftarrow s] \equiv (\Pi x\colon C[y \leftarrow s])D[y \leftarrow s]$. Apply Sub to $\Gamma, y\colon B, \Delta, x\colon C \vdash D$ type to obtain $\Gamma, \Delta[y \leftarrow s], x\colon C[y \leftarrow s] \vdash D[y \leftarrow s]$ type.
- Case 4: $A[y \leftarrow s] \equiv S[z_i \leftarrow t_i[y \leftarrow s]]$. Apply Sub to each typing derivation using $(C_i[z_1 \leftarrow t_1, \ldots])[y \leftarrow s] \equiv C_i[z_1 \leftarrow t_1[y \leftarrow s], \ldots]$.

Group III: The Reflection and Equality Rules

- Reflection2: $\dfrac{\Gamma \vdash t\colon A}{\Gamma \vdash A \text{ type}}$
  The sub-derivation $\pi'\colon \Gamma \vdash t\colon A$ satisfies $|\pi'| < |\pi|$. Apply Lemma B.2 (IH) to π'.
  - Lemma B.2 Case 1 ($t \equiv x$, $x\colon A \in \Gamma$): Let $\Gamma \equiv \Gamma_0, x\colon A, \Gamma_1$. Context Γ was formed by **??** from $\Gamma_0 \vdash A$ type. By applying Thin, $\Gamma \vdash A$ type is derivable by a shorter derivation. Apply the IH.
  - Lemma B.2 Case 2 ($t \equiv (\lambda x\colon B)s$, $A \equiv (\Pi x\colon B)C$): From $\Gamma, x\colon B \vdash s\colon C$, apply Reflection2 (strictly smaller height) to get $\Gamma, x\colon B \vdash C$ type. This is Case 3.
  - Lemma B.2 Case 3 ($t \equiv \text{App}([x\colon B]C, f, r)$, $A \equiv C[x \leftarrow r]$): From $\Gamma, x\colon B \vdash C$ type and $\Gamma \vdash r\colon B$, apply Sub to obtain $\Gamma \vdash C[x \leftarrow r]$ type by a shorter derivation. Apply the IH.
  - Lemma B.2 Case 4 ($t \equiv (\forall x\colon B)p$, $A \equiv \text{Prop}$): This is Case 1.
  - Lemma B.2 Case 5 ($t \equiv F[z_i \leftarrow t_i]$, $A \equiv D[z_i \leftarrow t_i]$): Apply Reflection2 to $\Gamma \vdash F[z_i \leftarrow t_i]\colon D[z_i \leftarrow t_i]$ (shorter) to get $\Gamma \vdash A$ type. Apply the IH.
- Refl-Typ: $\dfrac{\Gamma \vdash A = A}{\Gamma \vdash A \text{ type}}$
  A is syntactically of forms (1)-(4). We proceed by inner induction on the derivation of $\Gamma \vdash A = A$:
  - If derived from $\Gamma \vdash A$ type, it is strictly shorter; apply the outer IH.
  - If via Π-Equ: From $\Gamma, x\colon B \vdash C = C$, apply Refl-Typ (IH) to extract $\Gamma, x\colon B \vdash C$ type. This is Case 3.
  - If via Prop-Equ: From $\Gamma \vdash p = p\colon \text{Prop}$, apply Refl-Obj to extract $\Gamma \vdash p\colon \text{Prop}$. This is Case 2.
  - If via $\forall$-Elim: $\Gamma \vdash \text{Proof}((\forall x\colon B)p) = (\Pi x\colon B)\text{Proof}(p)$. If $A \equiv \text{Proof}((\forall x\colon B)p)$, it is Case 2. If $A \equiv (\Pi x\colon B)\text{Proof}(p)$, it is Case 3.

□

Lemma B.2. *Given a derivable sequent of the form $\Gamma \vdash t : A$ there are 5 possible cases:*

1. *there exists a variable x in $Var(\Gamma)$, such that t equals x;*
2. *t is equal to $(\lambda x : B)s$ and A is equal to $(\Pi x : B)C$ for some pre-type-expressions B, C, a pre-object-expression s and variable x;*
3. *t is equal to $App([x : B]C, s, r)$ and A is equal to $C[x \leftarrow r]$ for some pre-type-expressions B, C, pre-object-expressions s, r and variable x;*
4. *t is equal to $(\forall x : B)s$ and A is equal to Prop for some pre-type-expression B, pre-object-expression s and variable x;*
5. *there exist an operator symbol F in Σ with an introductory rule $y_1 : B_1, \ldots, y_m : B_m \vdash F : B$ and sequents*

$$\begin{aligned} &\Gamma \vdash t_1 : B_1 \\ &\Gamma \vdash t_2 : B_2[y_1 \leftarrow t_1] \\ &\quad\vdots \\ &\Gamma \vdash t_m : B_m[y_1 \leftarrow t_1, \ldots, y_{m-1} \leftarrow t_{m-1}] \end{aligned}$$

such that t is equal to $F[y_1 \leftarrow t_1, \ldots, y_m \leftarrow t_m]$.

Proof. By simultaneous induction on $|\pi|$ with Lemma B.1.

Group I: Direct Term Introduction Rules

- Ass: $\dfrac{\Gamma_0, x : A, \Delta \text{ ok}}{\Gamma_0, x : A, \Delta \vdash x : A}$
  $t \equiv x \in \text{Var}(\Gamma)$. This is Case 1.
- Π-Intro: $\dfrac{\Gamma, x : B \vdash s : C}{\Gamma \vdash (\lambda x : B)s : (\Pi x : B)C}$
  $t \equiv (\lambda x : B)s$ and $A \equiv (\Pi x : B)C$. The premise is a strict sub-derivation. This is Case 2.
- Π-Elim: $\dfrac{\Gamma \vdash f : (\Pi x : B)C \quad \Gamma \vdash r : B \quad \Gamma, x : B \vdash C \text{ type}}{\Gamma \vdash \text{App}([x : B]C, f, r) : C[x \leftarrow r]}$
  $t \equiv \text{App}([x : B]C, f, r)$ and $A \equiv C[x \leftarrow r]$. Premises are strict sub-derivations. This is Case 3.
- $\forall$-Intro: $\dfrac{\Gamma, x : B \vdash p : \text{Prop}}{\Gamma \vdash (\forall x : B)p : \text{Prop}}$
  $t \equiv (\forall x : B)p$ and $A \equiv \text{Prop}$. This is Case 4.
- Introductory Rule: For an operator $F \in \Sigma$,
  $t \equiv F[z_i \leftarrow t_i]$ with all premises as strict sub-derivations. This is Case 5.

Group II: Context-Structural Rules

These preserve the syntactic form of t; only the context and type representative change.

- EQU: $\dfrac{\Delta = \Gamma \quad \Delta \vdash t : A}{\Gamma \vdash t : A}$
  Apply the IH to $\Delta \vdash t : A$. Transfer each sub-derivation from context Δ to Γ using EQU (and CEQU where necessary). All cases transfer safely.

- THIN: $\dfrac{\Gamma, \Delta \vdash t : A \quad \Gamma \vdash B \text{ type}}{\Gamma, y : B, \Delta \vdash t : A}$
  Apply the IH to $\Gamma, \Delta \vdash t : A$. Transfer sub-derivations using THIN (using CONT-THIN for extended contexts, valid by Prop. **??**). All cases transfer.

- SUB: $\dfrac{\Gamma, y : B, \Delta \vdash t : A \quad \Gamma \vdash s : B}{\Gamma, \Delta[y \leftarrow s] \vdash t[y \leftarrow s] : A[y \leftarrow s]}$
  Apply the IH to $\Gamma, y : B, \Delta \vdash t : A$. Substitution is transparent to the outermost constructor of t. If $t \equiv x \equiv y$, then $t[y \leftarrow s] \equiv s$. Extend $\Gamma \vdash s : B$ by THIN and apply the IH to $\Gamma \vdash s : B$ (smaller height) to recursively determine the form of s.

Group III: Type-Conversion and Equality Rules

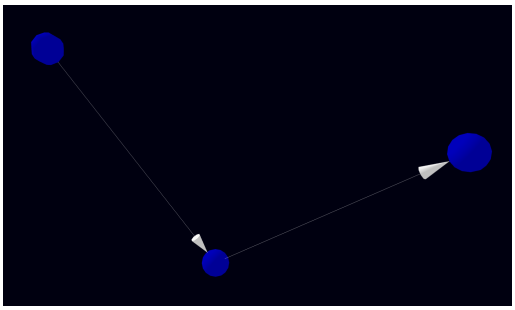
- CONV1: $\dfrac{\Gamma \vdash A' = A \quad \Gamma \vdash t : A'}{\Gamma \vdash t : A}$
  Apply the IH to $\Gamma \vdash t : A'$ (smaller height). The syntactic form of t is unchanged; we propagate the type claim from A' to A using provable equality $\Gamma \vdash A' = A$ and TRANS-TYP. For example, in Case 2, A' is provably equal to $(\Pi x : B)C$, and by transitivity, A is provably equal to $(\Pi x : B)C$.

- REFL-OBJ: $\dfrac{\Gamma \vdash t = t : A}{\Gamma \vdash t : A}$
  By inner induction on the derivation of term equality. If derived from $\Gamma \vdash t : A$, it is shorter (apply outer IH). If derived by TRANS-OBJ, the companion term-equality lemma determines the form. If derived by β-RULE or η-RULE, t maps directly to Case 3 or Case 2, respectively.

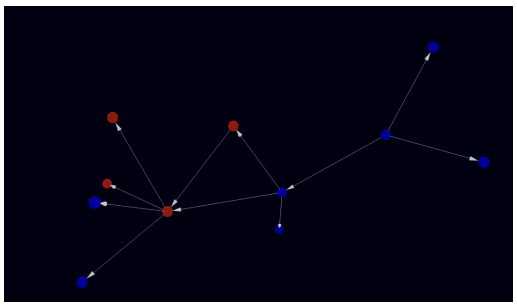
□

Appendix C

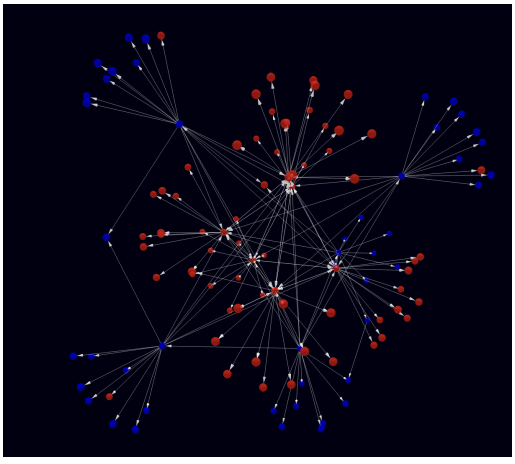
Figures



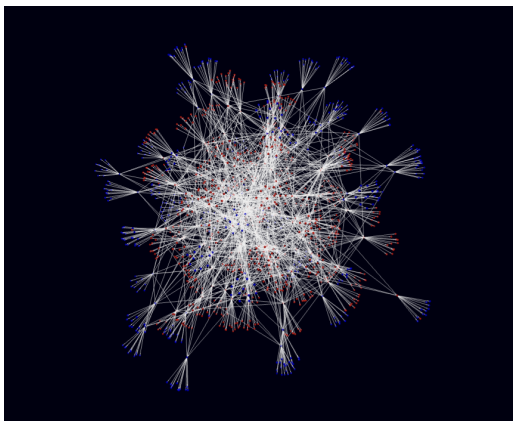
(a) The G_1 subcategory



(b) The G_2 subcategory



(c) The G_3 subcategory



(d) The G_4 subcategory

Figure C.1: Overview of the G_1 through G_4 subcategories.

Bibliography

[1] Alfred V. Aho and Alfred V. Aho (eds.), *Compilers: Principles, techniques, & tools*, 2nd ed ed., Pearson/Addison Wesley, Boston, 2007.

[2] Marc Bezem, Thierry Coquand, Peter Dybjer, and Martín Escardó, *A Note on Generalized Algebraic Theories and Categories with Families*, March 2021.

[3] Guillaume Bruneire and Peter LeFanu Lumsdaine, *Initiality for Martin-Löf type theory*, September 2020.

[4] John Cartmell, *Generalized Algebraic Theories and Contextual Categories*, Ph.D. thesis, Oxford University, 1978.

[5] ———, *Generalised algebraic theories and contextual categories*, Annals of Pure and Applied Logic **32** (1986), 209–243.

[6] Thierry Coquand and Gérard Huet, *The calculus of constructions*, Information and Computation **76** (1988), no. 2, 95–120.

[7] Thierry Coquand and Christine Paulin, *Inductively defined types*, COLOG-88 (Berlin, Heidelberg) (Per Martin-Löf and Grigori Mints, eds.), Springer, 1990, pp. 50–66.

[8] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer, *The Lean Theorem Prover (System Description)*, Automated Deduction - CADE-25 (Cham) (Amy P. Felty and Aart Middeldorp, eds.), Springer International Publishing, 2015, pp. 378–388.

[9] Kefan Dong and Tengyu Ma, *STP: Self-play LLM Theorem Provers with Iterative Conjecturing and Proving*, March 2025.

[10] Gilles Dowek, *The undecidability of typability in the Lambda-Pi-calculus*, Typed Lambda Calculi and Applications (Berlin, Heidelberg) (Marc Bezem and Jan Friso Groote, eds.), Springer, 1993, pp. 139–145.

[11] Carlos Fernández Lorán, *Implica at main · CarlosFerLo/implica*.

[12] William Alvin Howard, *The formulas-as-types notion of construction.*, To H. B. Curry : Essays on Combinatory Logic, Lambda Calculus, and Formalism (1980).

[13] Laurent Lafforgue, *Some possible roles for AI of Grothendieck topos theory*, 2022.

[14] ———, *Some sketches for a topos-theoretic AI*, 2024.

[15] F. William Lawvere, *Quantifiers and Sheaves*, Actes du Congrès International des Mathèmaticiens **1** (1971), 329–334.

[16] Logical Intelligence, *EBM vs. LLMs: Our Kona EBM a 96% vs. 2% Sudoku Benchmark*, https://logicalintelligence.com/blog/energy-based-model-sudoku-demo.

[17] Saunders Mac Lane, *Categories for the Working Mathematician*, Springer-Verlag, 1971.

[18] ———, *Sheaves in geometry and logic: A first introduction to topos theory*, Universitext, Springer-Verlag, New York, 1992.

[19] Tambet Matiisen, Avital Oliver, Taco Cohen, and John Schulman, *Teacher-Student Curriculum Learning*, November 2017.

[20] Christine Paulin-Mohring, *Inductive definitions in the system Coq rules and properties*, Typed Lambda Calculi and Applications (Berlin/Heidelberg) (Marc Bezem and Jan Friso Groote, eds.), vol. 664, Springer-Verlag, 1993, pp. 328–345.

[21] Thomas Streicher, *Semantics of Type Theory Correctness, Completeness and Independence Results*, 1991.

[22] Sainbayar Sukhbaatar, Zeming Lin, Ilya Kostrikov, Gabriel Synnaeve, Arthur Szlam, and Rob Fergus, *Intrinsic Motivation and Automatic Curricula via Asymmetric Self-Play*, April 2018.

[23] A. S. Troelstra and H. Schwichtenberg, *Basic Proof Theory*, 2 ed., Cambridge Tracts in Theoretical Computer Science, Cambridge University Press, Cambridge, 2000.

[24] Vladimir Voevodsky, *Models, Interpretations and the Initiality Conjectures*, (2017).

[25] Mingzhe Wang and Jia Deng, *Learning to Prove Theorems by Learning to Generate Theorems*, October 2020.

[26] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu, *A Comprehensive Survey on Graph Neural Networks*, IEEE Transactions on Neural Networks and Learning Systems **32** (2021), no. 1, 4–24.

[27] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun, *Graph neural networks: A review of methods and applications*, AI Open **1** (2020), 57–81.